

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
по курсу
«Data Science Pro»

**Тема: «Прогнозирование конечных свойств новых материалов
(композиционных материалов)»**

Слушатель

Васильева Галина Львовна

Москва, 2025

Оглавление

Введение.....	3
1. Аналитическая часть.....	5
1.1 Постановка задачи.....	5
1.2 Описание используемых методов.....	8
1.2.1 Описательная статистика и выявление аномалий.....	9
1.2.2 Нормализация данных.....	12
1.2.3 Прогнозные модели и их метрики.....	14
1.3 Разведочный анализ данных.....	21
2. Практическая часть.....	27
2.1 Предобработка данных.....	27
2.1.1 Очистка данных от выбросов и анализ результатов.....	27
2.1.2 Нормализация значений переменных.....	29
2.2 Разработка, обучение и тестирование моделей прогнозирования модуля упругости при растяжении.....	31
2.3 Разработка, обучение и тестирование моделей прогнозирования прочности при растяжении.....	37
2.4 Создание нейронной сети для рекомендации соотношения «матрица – наполнитель».....	39
2.5 Разработка веб-приложения.....	41
2.6 Создание удалённого репозитория и загрузка результатов.....	42
Заключение.....	44
Библиографический список.....	45

Введение

Развитие промышленности обуславливает необходимость создания и применения новых видов материалов, от которых ожидают свойств, соответствующих новым требованиям науки и производства.

Композиционные материалы уже успели зарекомендовать себя как материалы будущего, обладающие большим потенциалом. Композиционные материалы имеют комплекс свойств и ряд особенностей, которые отличают их от традиционных конструкционных материалов (например, металлических сплавов) и в совокупности открывают широкие возможности как для совершенствования конструкций самого разнообразного назначения, так и для разработки новых конструкций и технологических процессов [12, С.4].

Современную эпоху можно назвать веком полимеров и композиционных материалов.

Основные задачи структурной механики композитов включают прогнозирование свойств композита и исследование процессов деформирования и разрушения конструкций из композитных материалов.

Существует три основных подхода к прогнозированию механических свойств полимеров и композитов, в том числе при воздействии на них внешней среды. Первый подход основан на принципе аналогий, второй подход использует экстраполяцию результатов натурных испытаний с помощью математических моделей, третий подход заключается в назначении минимального срока службы материалов в естественных условиях с постепенным продлением этого периода. [7]

Каждый из приведенных методов имеет свои преимущества и ограничения в использовании и может быть эффективно дополнен другими.

Предметом исследований в работе выступают математические модели прогнозирования эксплуатационных свойств композитов, в качестве *объекта*

исследования используется набор данных – физико-механических характеристик композиционных материалов и их компонентов.

Целью написания работы является разработка моделей для прогнозирования прочностных характеристик композита на основе имеющихся данных о результатах испытаний и свойствах его компонентов.

Для достижения поставленной цели решается ряд *исследовательских задач*:

- проведение анализа исходной информации: значений показателей, характеризующих свойства композита и его компонентов;
- осуществление предобработки данных, включая, при необходимости, их очистку от выбросов, пропусков, нормализацию и стандартизацию;
- поиск и оценка моделей машинного обучения, включая нейросетевые, для создания прогнозов конечных свойств композитов на их основе;
- создание приложения с графическим интерфейсом или интерфейсом командной строки, которое будет выдавать прогноз, смоделированный ранее.
- создание репозитория в GitHub и размещение в нем файлов исследования.

Структурно работа представлена аналитической и практической частями, введением, заключением, библиографическим списком.

При написании работы использованы *методы* корреляционно-регрессионного анализа, описательной статистики, дерева решений, k-ближайших соседей, градиентного бустинга, нейросетевого моделирования, графического анализа и другие. Исследования проводились посредством использования высокоуровневого языка программирования Python в программных средах Jupyter-ноутбук, Visual Studio.

1. Аналитическая часть

1.1 Постановка задачи

Для решения поставленных во введении задач написания выпускной квалификационной работы с целью построения прогнозных моделей прочностных характеристик композиционных материалов необходимо провести анализ имеющихся в распоряжении исследователя данных. Анализ и оценка таких данных включает в концепции Data Science, как деятельности, связанной с анализом данных и поиском лучших решений на их основе, разведочный и подтверждающий виды анализа. Первый этап связан с преобразованием данных наблюдений и способами их наглядного представления, позволяющими выявить внутренние закономерности, проявляющиеся в данных. Цели разведочного анализа на первом этапе включают максимальное «проникновение» в данные, выявление основных структур, выбор наиболее важных переменных, обнаружение отклонений и аномалий, проверку основных гипотез и разработку начальных моделей.

На втором этапе - подтверждающем анализе - применяются традиционные статистические методы оценки параметров и проверки гипотез.

Специалисты компании Amazon выделяют четыре вида анализа данных, который реализуется в рамках Data Science:

1. Описательный анализ (Descriptive analysis) изучает данные, чтобы получить представление о том, что произошло или происходит в среде данных.
2. Диагностический анализ (Diagnostic analysis) предполагает глубокое погружение или подробное изучение данных для понимания того, почему что-то произошло. При этом над заданным набором данных могут быть выполнены множественные преобразования, чтобы обнаружить уникальные закономерности в каждом из этих методов.

3. Предиктивный анализ (Predictive analysis) использует исторические данные для составления точных прогнозов относительно закономерностей данных, которые могут возникнуть в будущем.

4. Предписывающая аналитика (Prescriptive analysis) выводит предиктивные данные на новый уровень. Она не только предсказывает, что, скорее всего, произойдет, но и предлагает оптимальный ответ на это. Она может анализировать потенциальные последствия различных вариантов и рекомендовать наилучший курс действий. [4, С.7-9]

В нашем случае исходные данные представлены в двух Excel таблицах:

1. X_br.xlsx с размерностью 1024 строки, 11 столбцов (включая строку с наименованиями столбцов и столбец с нумерацией строк) и значениями следующих показателей-характеристик физико-механических свойств композитов и их компонентов по столбцам: «Соотношение матрица-наполнитель», «Плотность, кг/м³», «модуль упругости, ГПа», «Количество отвердителя, м.-%», «Содержание эпоксидных групп, %₂», «Температура вспышки, С₂», «Поверхностная плотность, г/м²», «Модуль упругости при растяжении, ГПа», «Прочность при растяжении, МПа», «Потребление смолы, г/м²»;

2. X_nir.xlsx с размерностью 1041 строка, 4 столбца (включая строку с наименованиями столбцов и столбец с нумерацией строк) и значениями следующих показателей-характеристик физико-механических свойств композитов и их компонентов по столбцам: «Угол нашивки, град», «Шаг нашивки», «Плотность нашивки».

В соответствии с условиями данные таблиц (df 1 и df 2) загружены с <https://drive.google.com> в jupyter-ноутбук (рис.1).

Данные объединены по индексу, тип объединения – INNER, с получением итогового датасета размерностью 1023 строки и 13 столбцов (рис.2).

Столбец Unnamed, не содержащий значимой информации, удаляется из датасета.



```
[5]: # Загружаем архивные файлы характеристик композиций (датасеты) с URL с помощью инструмента загрузки больших файлов
!pip install gdown
import gdown
url = 'https://drive.google.com/uc?id=1B1s5gBlvGU81H9GGolLQVw_S0i-vyNf2'
output = 'hw_data_composite.zip'
gdown.download(url, output, quiet=False)

Requirement already satisfied: gdown in c:\users\user\anaconda3\lib\site-packages (5.2.0)
Requirement already satisfied: beautifulsoup4 in c:\users\user\anaconda3\lib\site-packages (from gdown) (4.12.3)
Requirement already satisfied: filelock in c:\users\user\anaconda3\lib\site-packages (from gdown) (3.17.0)
Requirement already satisfied: requests[socks] in c:\users\user\anaconda3\lib\site-packages (from gdown) (2.32.3)
Requirement already satisfied: tqdm in c:\users\user\anaconda3\lib\site-packages (from gdown) (4.67.1)
Requirement already satisfied: soupsieve>1.2 in c:\users\user\anaconda3\lib\site-packages (from beautifulsoup4->gdown) (2.5)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\user\anaconda3\lib\site-packages (from requests[socks]->gdown) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\anaconda3\lib\site-packages (from requests[socks]->gdown) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\anaconda3\lib\site-packages (from requests[socks]->gdown) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\user\anaconda3\lib\site-packages (from requests[socks]->gdown) (2025.4.26)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in c:\users\user\anaconda3\lib\site-packages (from requests[socks]->gdown) (1.7.1)
Requirement already satisfied: colorama in c:\users\user\anaconda3\lib\site-packages (from tqdm->gdown) (0.4.6)

Downloading...
From: https://drive.google.com/uc?id=1B1s5gBlvGU81H9GGolLQVw_S0i-vyNf2
To: c:\Users\User\Desktop\Data Science\проект программ\hw_data_composite.zip
100% | 178k/178k [00:00<00:00, 2.90MB/s]
```

```
[5]: 'hw data composite.zip'
```

```
[7]: # Извлекаем 2 excel-файла данных из архива Zip и выводим таблицы с показателями на экран
import pandas as pd
from zipfile import ZipFile
with ZipFile("hw_data_composite.zip", "r") as myzip:
    myzip.extractall()
df1 = pd.read_excel('X_bp.xlsx')
df2 = pd.read_excel('X_nup.xlsx')
```

```
[8]: df1
```

[8]:

Unnamed: 0	Соотношение матрица-наполнитель	Плотность, кг/м3	модуль упругости, ГПа	Количество отвердителя, м.м%	Содержание эпоксидных групп, %_2	Температура вспыхши, C_2	Поверхностная плотность, г/м2	Модуль упругости при растяжении, ГПа	Прочность при растяжении, МПа	Потребление смолы, г/м2	
0	0	1.857143	2030.000000	738.736842	30.000000	22.267857	100.000000	210.000000	70.000000	3000.000000	220.000000
1	1	1.857143	2030.000000	738.736842	50.000000	23.750000	284.615385	210.000000	70.000000	3000.000000	220.000000
2	2	1.857143	2030.000000	738.736842	49.900000	33.000000	284.615385	210.000000	70.000000	3000.000000	220.000000
3	3	1.857143	2030.000000	738.736842	129.000000	21.250000	300.000000	210.000000	70.000000	3000.000000	220.000000
4	4	2.771331	2030.000000	753.000000	111.860000	22.267857	284.615385	210.000000	70.000000	3000.000000	220.000000
...	...	...	...	...	...	...	...	...	...	...	...
1018	1018	2.271346	1952.087902	912.855545	86.992183	20.123249	324.774576	209.198700	73.090961	2387.292495	125.007669
1019	1019	3.444022	2050.089171	444.732634	145.981978	19.599769	254.215401	350.660830	72.920827	2360.392784	117.730099
1020	1020	3.280604	1972.372865	416.836524	110.533477	23.957502	248.423047	740.142791	74.734344	2662.906040	236.606764
1021	1021	3.705351	2066.799773	741.475517	141.397963	19.246945	275.779840	641.468152	74.042708	2071.715856	197.126067
1022	1022	3.808020	1890.413468	417.316232	129.183416	27.474763	300.952708	758.747882	74.309704	2856.328932	194.754342

1023 rows × 11 columns

1023 rows x 11 columns

```
[12]: df2
```

[12]:	Unnamed: 0	Угол нашивки, град	Шаг нашивки	Плотность нашивки
0	0	0	4.000000	57.000000
1	1	0	4.000000	60.000000
2	2	0	4.000000	70.000000
3	3	0	5.000000	47.000000
4	4	0	5.000000	57.000000
...	...	...	...	...
1035	1035	90	8.088111	47.759177
1036	1036	90	7.619138	66.931932
1037	1037	90	9.800926	72.858286
1038	1038	90	10.079859	65.519479
1039	1039	90	9.021043	66.920143

1040 rows x 4 columns

Рисунок 1- Исходные данные двух Excel таблиц

[8]: # В соответствии с заданием объединяем таблицы по индексу, используя метод INNER.
df = df1.merge(df2, how='inner', on='Unnamed: 0')

Столбец "Unnamed:0", не содержащий значимой информации, целесообразно удалить.
df = df.drop('Unnamed: 0', axis=1)
df

[8]:

	Соотношение матрица-наполнитель	Плотность, кг/м3	модуль упругости, ГПа	Количество отвердителя, м.%	Содержание эпоксидных групп, %_2	Температура всплышки, C_2	Поверхностная плотность, г/м2	Модуль упругости при растяжении, ГПа	Прочность при растяжении, МПа	Потребление смолы, г/м2	Угол нашивки, град	наш
0	1.857143	2030.000000	738.736842	30.000000	22.267857	100.000000	210.000000	70.000000	3000.000000	220.000000	0	4.00
1	1.857143	2030.000000	738.736842	50.000000	23.750000	284.615385	210.000000	70.000000	3000.000000	220.000000	0	4.00
2	1.857143	2030.000000	738.736842	49.900000	33.000000	284.615385	210.000000	70.000000	3000.000000	220.000000	0	4.00
3	1.857143	2030.000000	738.736842	129.000000	21.250000	300.000000	210.000000	70.000000	3000.000000	220.000000	0	5.00
4	2.771331	2030.000000	753.000000	111.860000	22.267857	284.615385	210.000000	70.000000	3000.000000	220.000000	0	5.00
...	...	...	...	...	...	...	...	...	...	...	...	...
1018	2.271346	1952.087902	912.855545	86.992183	20.123249	324.774576	209.198700	73.090961	2387.292495	125.007669	90	9.07
1019	3.444022	2050.089171	444.732634	145.981978	19.599769	254.215401	350.660830	72.920827	2360.392784	117.730099	90	10.56
1020	3.280604	1972.372865	416.836524	110.533477	23.957502	248.423047	740.142791	74.734344	2662.906040	236.606764	90	4.16
1021	3.705351	2066.799773	741.475517	141.397963	19.246945	275.779840	641.468152	74.042708	2071.715856	197.126067	90	6.31
1022	3.808020	1890.413468	417.316232	129.183416	27.474763	300.952708	758.747882	74.309704	2856.328932	194.754342	90	6.07

1023 rows x 13 columns

Рисунок 2- Датасет с характеристиками композитов и их компонентов для проведения анализа

В качестве результативных целевых характеристик выступают показатели модуля упругости при растяжении и прочности при растяжении, остальные показатели выступают как независимые переменные самих композитов или их компонентов. Выборка содержит показатели с дробными и целочисленными значениями, переменные с нулевыми значениями. В рамках проведения разведочного анализа данных переменные будут изучены более подробно с привлечением широкого арсенала методов исследования.

1.2 Описание используемых методов

В работе с полученной выборкой с целью получения прогнозных моделей традиционно используется этапность работы с данными, начиная с разведочного анализа данных с описательной статистикой и работой с возможными аномалиями, предобработки данных и заканчивая построением моделей.

При этом на всех этапах широко применяется визуализация данных, в том числе столбчатые диаграммы, линейные графики, таблицы.

Инструментами, позволяющими использовать те или иные методы анализа данных в программной среде, являются встроенные алгоритмы библиотек Pandas, Scikit-learn, Numpy, Scipy.stats, Matplotlib, Seaborn, TensorFlow.

1.2.1 Описательная статистика и выявление аномалий

Разведочный анализ данных – комплекс процедур для оценок данных (exploratory data analysis, EDA) включает непараметрическую и робастную статистику, визуализации, способствующие интегральному представлению данных и выявлению закономерностей в них.

Для представления результатов описательной статистики, кроме набора статистических показателей, широко используются таблицы и графики, которые помогают обобщенно оценить набор данных и выявить его закономерности.

Датасет сопровождается статистикой, включающей:

1) меры центральной тенденции (меры положения):

- минимальное, максимальное, среднее значения,
- медиана,
- мода,

2) квантильные оценки (квартили 25%,50%,75%);

3) меры вариации (меры разброса):

- размах,
- стандартное отклонение:

$$\sigma = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}, \quad (1)$$

где σ - величина стандартного отклонения,

n - количество наблюдений переменной x в выборке,

x_i - i -ое значение переменной,

$\bar{x}$ - среднее по выборке.

- коэффициент вариации (соотношение стандартного отклонения и среднего значения).

Все метрики рассчитываются для каждой переменной входного датасета.

«Выпадающие» точки, или аномалии, в наборе данных являются одной из самых опасных проблем, с которой сталкиваются почти все проекты по работе с данными. Реальные наборы данных всегда содержат некоторые «нетиповые» сэмплы, поэтому ставится задача их обнаружить и соответствующим образом обработать. При обнаружении аномалий требуется учитывать механизм возникновения аномалий, контекст задачи и конкретный сценарий ее исполнения, а также во многом опираться на интуицию.

Выделяются аномалии двух типов - выбросы (outliers) и новинки (novelties). Выбросы - это аномальные или экстремальные точки данных, которые существуют в обучающих данных. Напротив, новинки - это новые или ранее не встречавшиеся случаи по сравнению с исходными (обучающими) данными. Подобрать удачное значение отсекающего коэффициента крайне сложно.

В качестве методов обнаружения и очистки от выбросов можно использовать:

- построение диаграммы «ящички с усами», на которых показаны минимальное, первое квартильное (Q_1), медианное, третье квартильное (Q_3) и максимальное значения набора данных. Выбросы отображаются в виде отдельных точек. Статистически этот стандартный метод определения выбросов с помощью IQR ($IQR = Q_3 - Q_1$) основан на построении так называемых «ограничительных усов» (fences): нижний ус = $Q_1 - 1,5 \times IQR$; верхний ус = $Q_3 + 1,5 \times IQR$.

Значения, выходящие за эти границы, считаются выбросами или аномалиями. Множитель 1,5 выбран эмпирически и является стандартным, однако в зависимости от области применения и чувствительности анализа, он может быть изменен на 2,0 или 3,0. Метод эффективно работает на асимметричных распределениях данных.

- построение гистограммы распределения (Histogram-based Outlier Score, HBOS) для каждой переменной с наложением графика нормального распределения, что позволяет наглядно оценить близость распределения к нормальному и асимметрию;

- метод процентилей выявляет выбросы на основе их положения в распределении данных. Он позволяет удалить точки данных, которые находятся ниже указанного нижнего процентиля или выше указанного верхнего процентиля. Однако, применительно к конкретному датасету данный метод может привести к значительному «обрезанию» выборки;

- модифицированный метод Z-показателя для идентификации выбросов с заменой их на NaN использует медиану и медианное абсолютное отклонение (MAD) вместо среднего и стандартного отклонения. Метод не требует, чтобы признак, на основе которого происходит поиск выбросов, был распределён нормально;

- исключение технических ошибок при включении данных в выборку на основе сравнения возможного диапазона значений отдельных переменных, характеризующих физико-механические свойства материалов и их компонентов, и их значений в датасете.

Корреляционная связь означает, что каждому значению одной переменной соответствует определенное математическое ожидание другой переменной.

Для изучения корреляции между переменными датасета эффективно используются такие методы, как:

- построение корреляционной матрицы с коэффициентами корреляции между парами значений изучаемых переменных в виде диапазона значений от -1,0 до 1,0 (значение (-1,0) указывает на идеальную отрицательную линейную зависимость; значение (1,0) указывает на идеальную положительную линейную зависимость; значение (0,0) указывает на отсутствие линейной зависимости или независимости между двумя переменными);

- визуализация с помощью диаграмм рассеяния (scatter plots), на которых отображаются взаимосвязи между двумя непрерывными переменными. Каждая точка на диаграмме представляет собой отдельный экземпляр данных со значениями признаков x и y соответственно. Диаграммы рассеяния используются для быстрого выявления потенциальных корреляций;

- визуализация посредством построения так называемых «тепловых карт» (heatmaps), которые отображают степень связи между двумя переменными в виде цветового кода, интенсивность цвета соответствует более высоким значениям коэффициента корреляции (цветовая шкала показана сбоку на карте).

1.2.2 Нормализация данных

Многие алгоритмы машинного обучения, используемые для получения прогнозных моделей, требуют нормализации входных признаков. Например, при использовании алгоритма SVM, а также линейной модели с L1 и L2 регуляризацией целевая функция предполагает, что все признаки центрированы в нуле и имеют дисперсии одного и того же порядка. Если признак имеет дисперсию большего порядка, он будет доминировать в целевой функции. Это может привести к тому, что модель будет работать плохо и не сможет учиться на других признаках.

Не требуют нормализации входных данных такие алгоритмы, как деревья принятия решений (и случайные леса), градиентный бустинг, наивный Байес. Напротив, требуют нормализации такие алгоритмы, как логистическая регрессия, SVM, нейронные сети, метод главных компонент.

После удаления выбросов с учетом сохранения репрезентативности выборки данные (п.1.2.1) целесообразно провести проверку на нормальность распределения значений переменных. Проверить это можно с использованием ранее рассмотренных методов графической визуализации или тестов, в том числе наиболее распространённых и надёжных из них:

1. Критерий Шапиро-Уилка специально разработан для проверки нормальности и лучше всего подходит для наборов данных малого и среднего размера (обычно до 2000 наблюдений). Если значения коэффициента значимости $\alpha > 0,05$, данные могут быть распределены нормально (нулевую гипотезу не удастся отклонить). В противном случае данные значительно отклоняются от нормального распределения (нулевая гипотеза отвергается).

2. Критерий Колмогорова-Смирнова (К-С) лучше всего подходит для больших наборов данных. Сравнивает эмпирическую функцию распределения выборки с кумулятивной функцией распределения нормального распределения (α - значение $> 0,05$: существенной разницы нет, и данные могли быть распределены нормально; в противном случае данные распределены неравномерно). Может быть слишком чувствительным при работе с очень большими выборками, выявляя незначительные отклонения как существенные.

В случае выявления ненормальности данных можно воспользоваться специальными методами, позволяющими преобразовать исходную «ненормальную статистику» в «нормальную».

К наиболее распространенным методам стандартизации данных относятся Z-оценка, минимаксная стандартизация, десятичное масштабирование.

Минимаксная нормализация линейно преобразует данные таким образом, что преобразованные значения находятся в интервале $[0, 1]$. Формула для Min-Max нормализации выглядит следующим образом:

$$x_{i \text{ norm}} = (x_i - x_{\min}) / (x_{\max} - x_{\min}), \quad (3)$$

где x_i – i -ое значение переменной,

$x_{\min}$ – минимальное значение переменной во всем наборе данных,

$x_{\max}$ – максимальное значение переменной во всем наборе данных,

$x_{i \text{ norm}}$ – нормализованное значение x_i -ое.

Выбор метода нормализации данных зависит от специфики нормализуемой переменной, в первую очередь, от вида распределения. В целом пайплайн

нормализации данных включает центрирование данных, масштабирование данных до заданного интервала, смещение масштабированных данных так, чтобы границы скорректированного интервала приходились на $[0..1]$.

В качестве центра (нулевого значения) для нормальных распределений берется среднее арифметическое, для распределений, отличных от нормального - медиана, так как она практически не чувствительна к выбросам и асимметрии распределения.

1.2.3 Прогнозные модели и их метрики

На выбор используемого алгоритма машинного обучения с целью построения прогнозной модели решающее влияние оказывают три группы факторов:

- структура и содержание решаемой задачи и ее соответствие в терминах задачи машинного обучения. Задачи обычно делятся на классификацию, регрессию, кластеризацию и т.д. В нашем случае имеет место задача регрессии;
- тип и структура имеющихся данных. Данные могут быть числовыми, категориальными, текстовыми или основанными на изображениях, и для каждого из них требуются различные методы предварительной обработки и моделирования;
- уровень сложности модели. Процесс выбора подходящей модели должен проходить постепенно. Лучше всего начать с простейших моделей в своей категории, таких как линейная или логистическая регрессия. Эти модели легко построить и интерпретировать, они быстро обучаются и обеспечивают надежный базис для дальнейших усовершенствований. Постепенное увеличение сложности модели позволяет определить, обеспечивают ли более сложные модели значительные улучшения.

В таблице 1 представлены основные виды алгоритмов моделей машинного обучения, с которыми работают в первую очередь при построении прогнозов.

Таблица 1- Основные алгоритмы моделирования

Алгоритм	Описание и особенности	Преимущества	Недостатки
Линейная регрессия (Linear Regression)	Моделирует отношения между непрерывными переменными. Большие датасеты с высокой размерностью	Хорошо работает с пропусками и выбросами, работает с категориальными и числовыми переменными	Предполагается линейная зависимость между признаками (плохо работает при нелинейных зависимостях), чувствительность к выбросам
Полиномиальная регрессия (Polynomial Regression)	Метод расширения линейной регрессии для моделирования сложных, нелинейных зависимостей между переменными. В ней есть класс Linear Regression для выполнения регрессии и класс Polynomial Features для генерации полиномиальных признаков	Легко интерпретируется, улучшает точность прогноза	Модель слишком сложна при больших степенях. Может становиться настолько гибкой, что начинает подстраиваться не только под основной тренд, но и под случайные флуктуации и шум, присутствующие именно в обучающей выборке
Дерево решений (Decision Tree)	Обучение с учителем, в процессе которого разделяются данные на ветви для принятия решения. работает как блок-схема, помогая принимать решения шаг за шагом, где: –внутренние узлы представляют собой тесты атрибутов; –ветви представляют значения атрибутов –конечные узлы представляют собой окончательные решения или прогнозы. Часто используется в кредитном скоринге, медицинской диагностике	Легко визуализировать, работает с категориальными и числовыми переменными	Легко переобучается, требуется большой объем данных
Случайный лес (Random Forest)	Создает множество деревьев. Каждое дерево анализирует разные случайные выборки данных, а их результаты объединяются путём голосования при классификации или усреднения при регрессии (среднее значение всех прогнозов деревьев), что делает этот метод ансамблевым. При построении каждого дерева оно не рассматривает все признаки (столбцы) сразу, а выбирает несколько признаков случайным образом, чтобы решить, как разделить данные. Это помогает деревьям отличаться друг от друга.	Работает с пропусками и выбросами, с категориальными и числовыми переменными	Требуется большой объем данных. Отнимает больше времени, чем деревья решений или линейные алгоритмы

Продолжение таблицы 1

К - ближайших соседей (K Nearest Neighbors)	Алгоритм обучения с учителем, который обычно используется для классификации, но может применяться и для задач регрессии. Он находит «к» ближайших точек данных (соседей) к заданным входным данным и делает прогнозы на основе класса большинства (для классификации) или среднего значения (для регрессии). является непараметрическим методом обучения на основе примеров. В алгоритме к-ближайших соседей k — это просто число, которое указывает алгоритму, сколько близлежащих точек или соседей нужно рассмотреть при принятии решения. При регрессии алгоритм, по-прежнему, ищет K-ближайших точек через евклидово расстояние, расстояние до Манхэттена, их сочетание. Вместо того, чтобы голосовать за класс при классификации, он вычисляет среднее значение этих K соседей, как прогнозируемое значение для новой точки.	Простой, интуитивно понятный, эффективный для классификации и регрессии. Хорошо работает в задачах рекомендации по продуктам, обнаружения аномалий, распознавания образов	Плохо работает на больших датасетах, чувствителен к нерелевантным признакам, вычислительно затратный
Градиентный бустинг (GBA)	Алгоритм, в котором каждая новая модель обучается минимизировать функцию потерь (среднеквадратическая ошибка или кросс-энтропия) предыдущей модели с помощью градиентного спуска. Прогнозы новой модели добавляются в ансамбль (прогнозы всех моделей), и процесс повторяется до тех пор, пока не будет достигнут критерий остановки. Ансамбль состоит из нескольких деревьев, каждое из которых обучается исправлять ошибки предыдущего. На первой итерации Дерево 1 обучается на исходных данных x. Алгоритм делает прогнозы, которые используются для вычисления ошибок. На второй итерации Дерево 2 обучается с использованием матрицы признаков x, а ошибки Дерева 1 используются в качестве меток. Это означает, что Дерево 2 обучается прогнозировать ошибки Дерева 1. Этот процесс продолжается для всех деревьев в ансамбле. Каждое последующее дерево обучается прогнозировать ошибки предыдущего дерева.	Высокая точность, эффективно находит нелинейные зависимости в данных различной природы	Легко переобучается, требуется большой объем данных

Продолжение таблицы 1

	Параметрами градиентного бустинга являются: –n.estimators: Здесь указывается количество деревьев (оценщиков), которые необходимо построить. Чем больше значение, тем выше производительность модели, но тем больше время вычислений; –скорость обучения: масштабирует вклад каждого дерева; –random_state: обеспечивает воспроизводимость результатов. Установка фиксированного значения для random_state гарантирует, что вы будете получать одни и те же результаты при каждом запуске модели; - max_features: параметр ограничивает количество признаков, которые каждое дерево может использовать для разделения. Это помогает предотвратить переобучение, ограничивая сложность каждого дерева и способствуя разнообразию в модели.		
Регрессия опорных векторов (SVR)	Обработка многомерных данных без их предварительного преобразования или понижения размерности. Разновидность метода опорных векторов (SVM), пытается найти функцию, которая наилучшим образом предсказывает непрерывное выходное значение для заданного входного значения. работает по принципу построения линии (в простых случаях) или поверхности (в более сложных ситуациях), которая наилучшим образом соответствует точкам данных. SVR может использовать как линейные, так и нелинейные ядра (функции). Выбор ядра зависит от характеристик данных и сложности задачи. Эти функции преобразуют входные данные в многомерное пространство. К распространённым ядрам относятся линейное, полиномиальное, радиально-базисное (RBF) и сигмовидное.	Подходит для проблем классификации и регрессии; хорошо работает с нелинейными зависимостями, в условиях малых датасетов и высокоразмерных данных	Вычислительно затратный, плохо работает на больших датасетах, неустойчив к шуму

Продолжение таблицы 1

	Модель SVR инициализируется с указанием таких гиперпараметров, как тип ядра, гамма, C и эпсилон: –kernel= rbf указывает на используемую функцию ядра. В SVR обычно используется ядро радиально-базисной функции (RBF); –gamma (например, 0,5) - это параметр радиально-базисного ядра. Он регулирует влияние каждого обучающего примера. Низкое значение означает «далеко», а высокое - «близко»; –C =10 (100) - это параметр регуляризации. Он обеспечивает компромисс между правильной классификацией обучающих примеров и максимизацией маржи функции принятия решений; –epsilon (например, 0,05, 0,1) - это параметр эпсилон в модели эпсилон-SVR. Он определяет эпсилон-полосу, в пределах которой функция потерь при обучении не налагает штрафных санкций.		
Нейронные сети (Neural Networks)	Состоят из ключевых компонентов: –нейроны, получающие входные данные. Каждый нейрон управляется пороговым значением и функцией активации; –связи между нейронами, которые передают информацию; –веса и смещения определяют силу и влияние связей; –функции распространения: обрабатывают и передают данные между слоями нейронов; –правило обучения: метод, который со временем корректирует веса и смещения для повышения точности. Состоят из слоев: 1. Входной слой: сеть получает входные данные. Каждый входной нейрон в слое соответствует признаку во входных данных. 2. Скрытые слои: состоят из нейронов, которые преобразуют входные данные в формат, понятный выходному слою. 3. Выходной слой: формирует выходные данные модели в нужном формате.	Хорошо работают с нелинейными зависимостями и высокоразмерными данными	Вычислительно затратные, трудно интерпретировать

Для оценки полученной модели используют различные метрики, основными из которых в задачах регрессии являются:

1. Среднеквадратическая ошибка (Mean Squared Error, MSE) - простая метрика, которая вычисляет разницу между фактическим и прогнозируемым значением (ошибку), возводит ее в квадрат, а затем выдает среднее значение всех ошибок. Применяется в случаях, когда требуется подчеркнуть большие ошибки и выбрать модель, которая дает меньше именно больших ошибок. Большие значения ошибок становятся заметнее за счет квадратичной зависимости.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (4)$$

где n - количество наблюдений, по которым строится модель, и количество прогнозов,

y_i - фактические значение зависимой переменной для i -го наблюдения,

$\hat{y}_i$ - значение зависимой переменной, предсказанное моделью.

Таким образом, можно сделать вывод, что MSE настроена на отражение влияния именно больших ошибок на качество модели.

Недостатком использования MSE является то, что если на одном или нескольких неудачных примерах, возможно, содержащих аномальные значения будет допущена значительная ошибка, то возведение в квадрат приведёт к ложному выводу, что вся модель работает плохо. С другой стороны, если модель даст небольшие ошибки на большом числе примеров, то может возникнуть обратный эффект - недооценка слабости модели.

Показатель RMSE является корнем MSE и полезен, поскольку помогает приблизить масштаб ошибок к фактическим значениям, что делает его более интерпретируемым.

2. Средняя абсолютная ошибка (MAE) измеряет среднее абсолютное отклонение между предсказанными и фактическими значениями целевой

переменной. Это интуитивно понятная метрика, которая представляет «типичную» ошибку модели в тех же единицах, что и целевая переменная:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (5)$$

3. Средняя абсолютная масштабированная ошибка (Mean absolute scaled error, MASE) - это показатель, который позволяет сравнивать две модели. Если поместить MAE_i для новой модели в числитель, а MAE_j для исходной модели в знаменатель, то полученное отношение и будет равно MASE. Если значение MASE меньше 1,0, то новая модель работает лучше, если MASE равно 1,0, то модели работают одинаково, а если значение MASE больше 1,0, то исходная модель работает лучше, чем новая модель. Формула для расчета MASE имеет вид:

$$MASE = \frac{MAE_i}{MAE_j} \quad (6)$$

MASE симметрична и устойчива к выбросам.

4. Коэффициент детерминации R^2 (Coefficient of determination) - это статистический показатель, который показывает долю дисперсии зависимой переменной, объяснённой с помощью регрессионной модели. Наиболее общей формулой для вычисления коэффициента детерминации является следующая:

$$R^2(y, \hat{y}) = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (7)$$

где $\bar{y}$ - среднее фактическое значение зависимой переменной.

Главным преимуществом коэффициента детерминации перед мерами, основанными на ошибках, является его инвариантность к масштабу данных. Кроме того, он всегда изменяется в диапазоне от $-\infty$ до 1,0.

Когда значение коэффициента близко к 0,0 (т.е. ошибка модели с переменными примерно равна ошибке модели только с константой), это

указывает на низкое соответствие модели данным, когда модель с переменными работает не лучше модели с константой.

Кроме этого, бывают ситуации, когда коэффициент R^2 принимает отрицательные значения (обычно небольшие). Это произойдёт, если ошибка модели среднего становится меньше ошибки модели с переменной. В этом случае оказывается, что добавление в модель с константой некоторой переменной только ухудшает её (т.е. регрессионная модель с переменной работает хуже, чем предсказание с помощью простой средней).

На практике используют следующую шкалу оценок. Модель, для которой $R^2 > 0,5$, является удовлетворительной. Если $R^2 > 0,8$, то модель рассматривается как очень хорошая. Значения, меньшие 0,5 говорят о том, что модель имеет низкую прогностическую ценность.

1.3 Разведочный анализ данных

Использование методов разведочного исследовательского анализа данных (Exploratory Data Analysis EDA) имеет решающее значение для понимания данных, выявления закономерностей и получения информации, которая может быть использована для дальнейшего анализа или принятия решений.

Как указывалось выше, это итеративная процедура, которая включает в себя обобщение, визуализацию и изучение информации для выявления закономерностей, аномалий и взаимосвязей, которые могут быть неочевидны на первый взгляд. Рассмотрим и реализуем важнейшие этапы разведочного анализа исходных данных о композитах и их компонентах.

В наборе наших данных 13 столбцов и 1023 строки, которые характеризуются следующими параметрами:

1. `df.info()` Общая информация : 12 столбцов с дробными значениями данных и 1 столбец - с целочисленными значениями. Пропусков значений нет.
2. `duplicates = df.duplicated()`

```
num_duplicates = duplicates.sum()
```

Дубликаты в представленных данных отсутствуют.

3. `df.isnull().sum()` Пропущенные значения отсутствуют.

4. `df.describe()` Описательная статистика данных по столбцам датасета включает такие показатели, как среднее значение, минимальное и максимальное значения, стандартное отклонение, 25%, 50% и 75% квантили (рис.3).

```
# Описательная статистика данных, представленных в полученной таблице-датасете
df.describe()
```

	Соотношение матрица-наполнитель	Плотность, кг/м3	модуль упругости, ГПа	Количество отвердителя, м.%	Содержание эпоксидных групп, %_2	Температура вспышки, C_2	Поверхностная плотность, г/м2	Модуль упругости при растяжении, ГПа	Прочность при растяжении, МПа	Потребление смолы, г/м2	Угол нашивки, град
count	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000	1023.000000
mean	2.930366	1975.734888	739.923233	110.570769	22.244390	285.882151	482.731833	73.328571	2466.922843	218.423144	44.252199
std	0.913222	73.729231	330.231581	28.295911	2.406301	40.943260	281.314690	3.118983	485.628006	59.735931	45.015793
min	0.389403	1731.764635	2.436909	17.740275	14.254985	100.000000	0.603740	64.054061	1036.856605	33.803026	0.000000
25%	2.317887	1924.155467	500.047452	92.443497	20.608034	259.066528	266.816645	71.245018	2135.850448	179.627520	0.000000
50%	2.906878	1977.621657	739.664328	110.564840	22.230744	285.896812	451.864365	73.268805	2459.524526	219.198882	0.000000
75%	3.552660	2021.374375	961.812526	129.730366	23.961934	313.002106	693.225017	75.356612	2767.193119	257.481724	90.000000
max	5.591742	2207.773481	1911.536477	198.953207	33.000000	413.273418	1399.542362	82.682051	3848.436732	414.590628	90.000000

Рисунок 3- Описательная статистика данных физико-механических свойств композитов и их компонентов

По результатам описательной статистики можно сделать следующие наблюдения:

- показатель "угол нашивки" принимает в данной выборке только два варианта значений (0° или 90°);
- в показателях "шаг нашивки" и "плотность нашивки" есть строки с нулевыми значениями;
- ряд показателей имеют существенный размах значений и большое стандартное отклонение.

Расчет коэффициентов вариации, как относительной меры изменчивости изучаемого признака, позволил обнаружить, что значения переменных «модуль упругости», «поверхностная плотность», «шаг нашивки» в изучаемом датасете высоко изменчивы:

```
df.apply(lambda x: x.std() / x.mean())
```

Соотношение матрица-наполнитель 0.311641
Плотность, кг/м3 0.037317
модуль упругости, ГПа 0.446305
Количество отвердителя, м.% 0.255908
Содержание эпоксидных групп, %_2 0.108176
Температура вспышки, C_2 0.143217
Поверхностная плотность, г/м2 0.582756
Модуль упругости при растяжении, ГПа 0.042534
Прочность при растяжении, МПа 0.196856
Потребление смолы, г/м2 0.273487
Угол нашивки, град 1.017256
Шаг нашивки 0.371559
Плотность нашивки 0.216100

Значения коэффициентов вариации больше 33% (0,33) свидетельствуют о высокой неоднородности информации и необходимости исключения аномалий (выбросов) или принадлежности части данных к другой группе объектов, в связи с чем необходимо учесть полученные характеристики и в рамках дальнейшего анализа исключить возможные ошибочные значения в данных переменных.

Наглядно продемонстрировать наличие выбросов в значениях переменных датасета позволяют инструменты визуализации:

- построение графика «ящик с усами» (рис. 4);
- гистограмм распределения для каждой переменной, которые наглядно позволяют оценить близость распределения к нормальному и асимметрию (рис.5).


```

plt.figure(figsize=(15,10))
plt.suptitle('"Ящики с усами" переменных- характеристик композитов', fontsize = 15)
sns.boxplot(data=df, orient="h")
plt.xticks(rotation=30, ha='right')
plt.show()
    
```

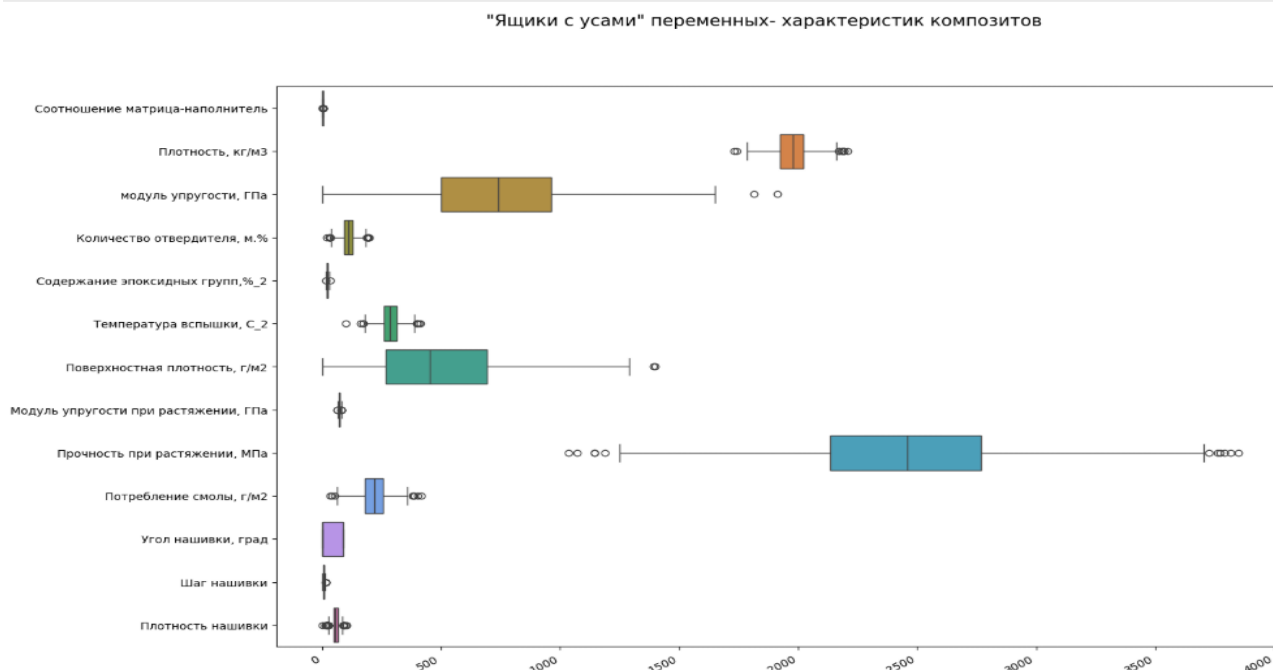


Рисунок 4 - График «Ящик с усами» переменных датасета

```

df.columns
column_names = df.columns
a = 5 # количество строк
b = 5 # количество столбцов
c = 1 # запуск plot counter
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize = (35,35))
plt.suptitle('Гистограммы распределения изучаемых переменных-характеристик', fontsize = 25)
for col in df.columns:
    plt.subplot(a, b, c)
    sns.histplot(data = df[col], kde=True, color = "darkblue")
    plt.ylabel(None)
    plt.title(col, size = 20)
    c += 1
    
```

Гистограммы распределения изучаемых переменных-характеристик

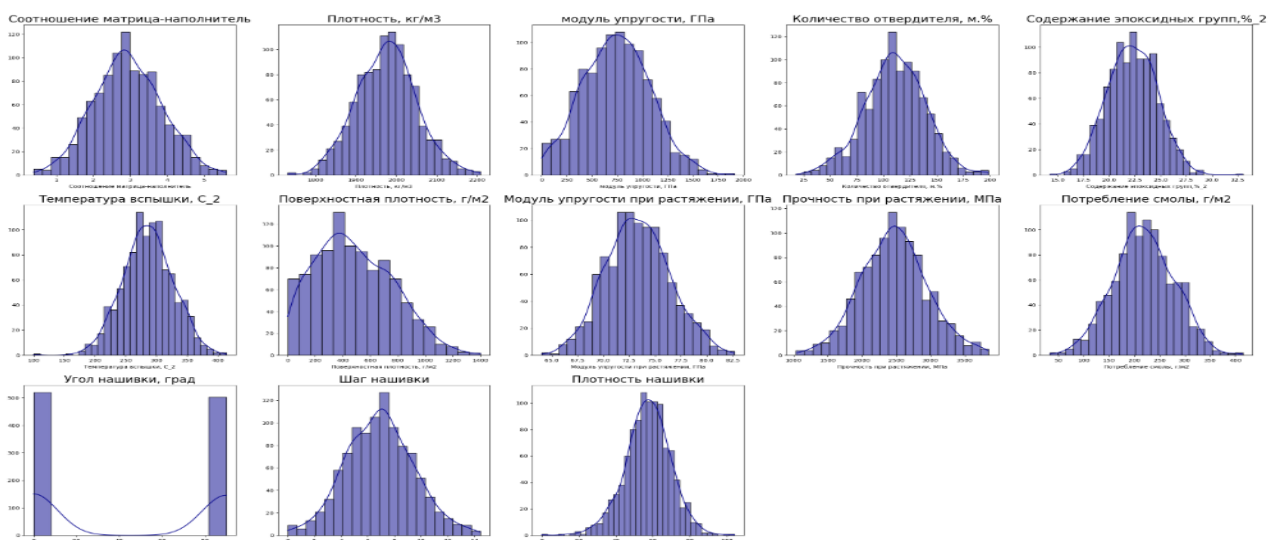


Рисунок 5 - Диаграммы распределения переменных датасета

Строка с нулевым значением показателей переменных «шаг нашивки» и «плотность нашивки» удалена из датасета, таким образом, количество строк уменьшилось до 1022 строк, количество столбцов 13:

```
df.loc[df['Плотность нашивки'] == 0]  
df_new = df[df['Плотность нашивки'] != 0]  
df_new.shape  
(1022, 13)
```

Визуально имеют место выбросы значений, а учитывая физико-механические свойства композитных материалов и используемых в них армирующих волокон, имеют место выбросы, прежде всего, в переменных «модуль упругости» и «поверхностная плотность», которые также демонстрируют асимметрию со смещением влево. Часть переменных по распределению значений близка к нормальному распределению.

На основе построения корреляционной матрицы для анализа типа и тесноты связи между переменными с помощью команды `df_new.corr()` получены значения коэффициентов корреляции в диапазоне от -0,075 до 0,105, свидетельствующие об отсутствии значимых линейных связей между переменными.

Для подтверждения вывода и визуализации использованы такие графически инструменты библиотеки Matplotlib, как диаграммы рассеяния (рис.6) и тепловая карта (рис. 7).

В целом по результатам разведочного анализа данных можно утверждать отсутствие линейной зависимости между переменными.

В датасете представлены результативные признаки-зависимые переменные по условию задачи (модуль упругости при растяжении, прочность при растяжении) и независимые переменные-факторы разных порядков, часть из которых, в свою очередь, может выступать как зависимая переменная для факторов более низкого уровня.

Для очистки датасета от выбросов значений переменных необходимо воспользоваться описанными выше методами.

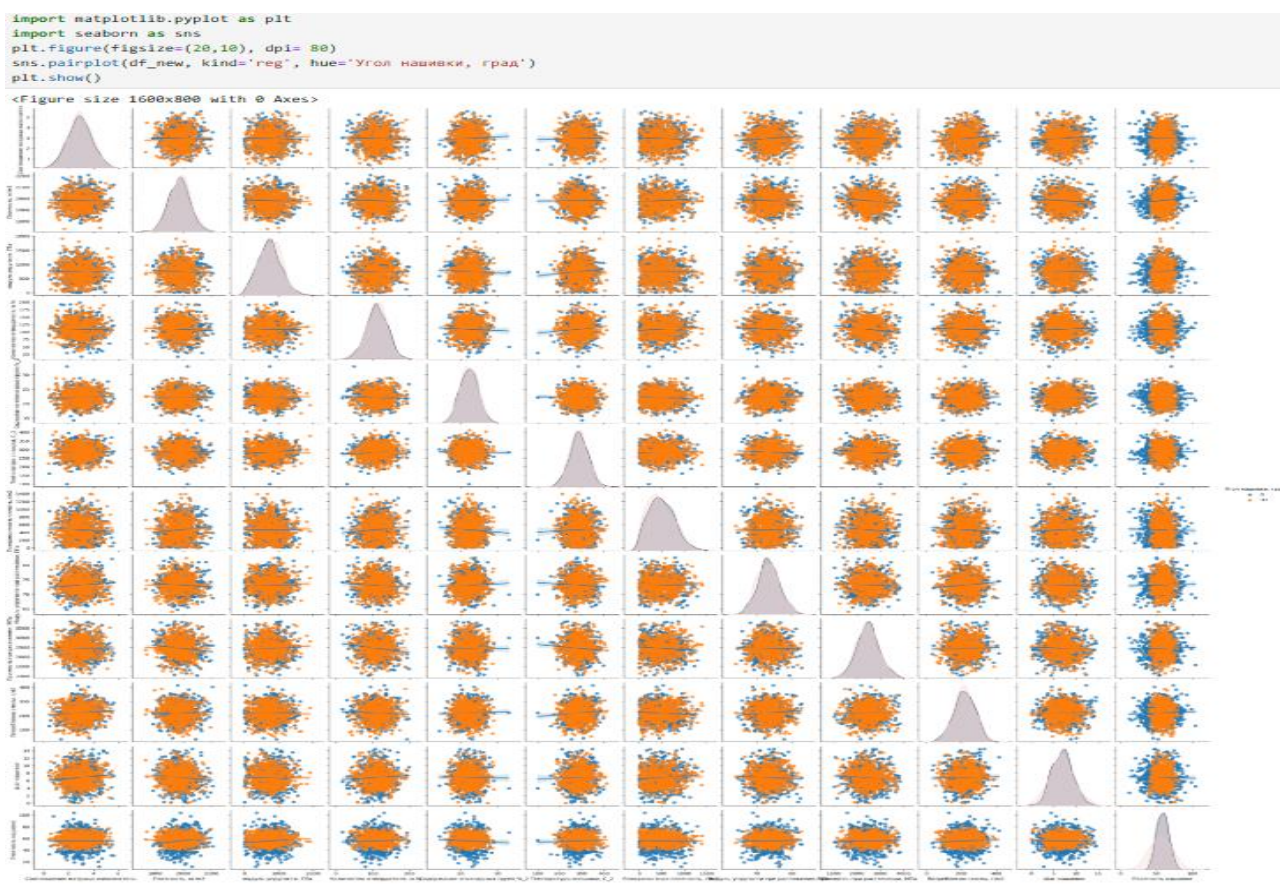


Рисунок 6- Диаграммы рассеяния значений переменных датасета

```
plt.figure(figsize=(15,6))
plt.suptitle("Тепловая карта" переменных- характеристик композитов', fontsize = 15)
sns.heatmap(df_corr().round(2), mask = np.triu(df_new.corr()), annot=True, cmap='plasma')
```

<Axes: >

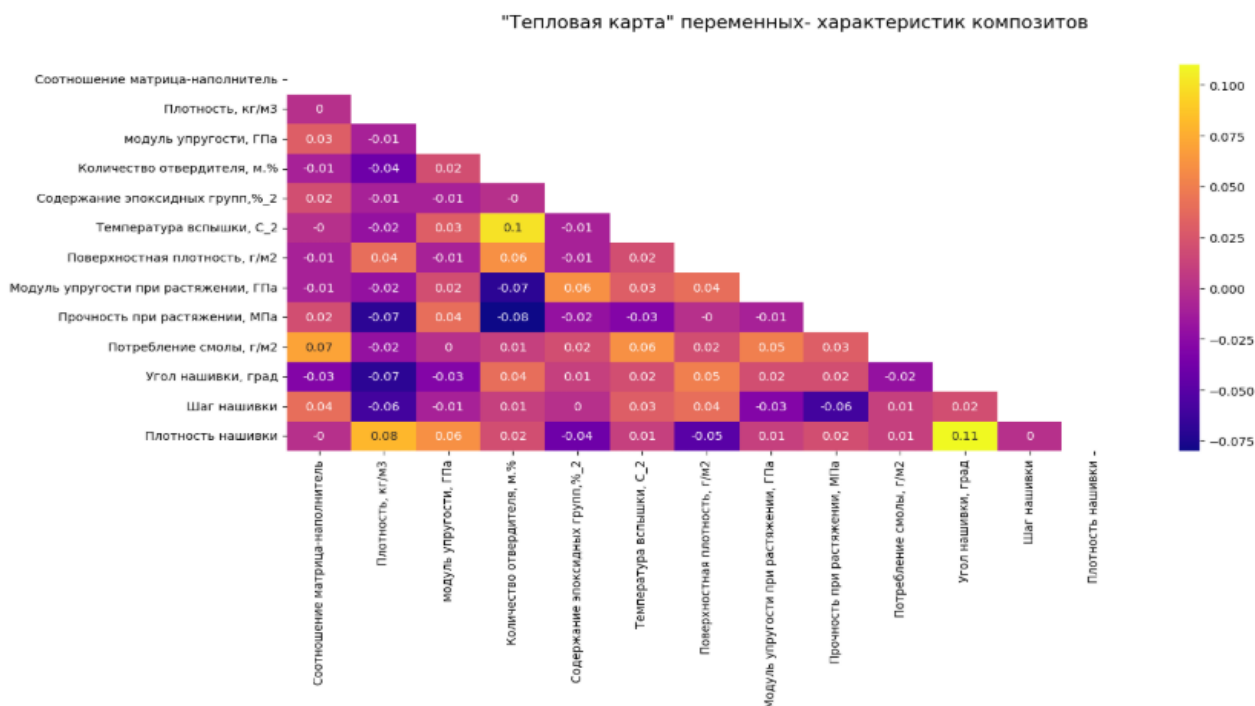


Рисунок 7 – Тепловая карта значений переменных датасета

2. Практическая часть

2.1 Предобработка данных

2.1.1 Очистка данных от выбросов и анализ результатов

В теоретической части работы были представлены наиболее часто используемые методы для обнаружения и удаления выбросов. Применительно к нашей выборке были использованы:

1. Метод процентилей с удалением точек данных, которые находятся ниже указанного нижнего процентиля или выше указанного верхнего процентиля, благодаря алгоритму:

```
def remove_outliers_percentile(df, columns, lower_percentile=0.5, upper_percentile=99.5):
    lower_bound = np.percentile(df[columns], lower_percentile, axis=0)
    upper_bound = np.percentile(df[columns], upper_percentile, axis=0)
    mask = (df_new[columns] >= lower_bound) & (df_new[columns] <= upper_bound)
    return df_new[mask]

columns_to_clean = ['Соотношение матрица-наполнитель', 'Плотность, кг/м3',
                    'модуль упругости, ГПа', 'Количество отвердителя, м.%',
                    'Содержание эпоксидных групп,%_2', 'Температура вспышки, С_2',
                    'Поверхностная плотность, г/м2', 'Модуль упругости при растяжении, ГПа',
                    'Прочность при растяжении, МПа', 'Потребление смолы, г/м2',
                    'Угол нашивки, град', 'Шаг нашивки', 'Плотность нашивки']
df_cleaned = remove_outliers_percentile(df_new, columns_to_clean)
```

В отношении анализируемого набора данных данный метод даже при минимальных процентах «обрезания» переменных в массиве намного сокращает общий объем данных.

2. Модифицированный метод Z-показателя для идентификации выбросов с заменой их на NaN без требований нормального распределения значений признака:

```
import numpy as np
from scipy import stats

columns_to_clean = ['Соотношение матрица-наполнитель', 'Плотность, кг/м3',
                    'модуль упругости, ГПа', 'Количество отвердителя, м.%',
                    'Содержание эпоксидных групп,%_2', 'Температура вспышки, С_2',
                    'Поверхностная плотность, г/м2', 'Модуль упругости при растяжении, ГПа',
                    'Прочность при растяжении, МПа', 'Потребление смолы, г/м2',
                    'Угол нашивки, град', 'Шаг нашивки', 'Плотность нашивки']

def remove_outliers_robust_zscore(df_new, columns, threshold=3):
    median = df_new[columns].median()
    mad = stats.median_abs_deviation(df_new[columns])
    z_scores = 0.6745 * (df_new[columns] - median) / mad
    mask = (z_scores < -threshold) | (z_scores > threshold)
    return df_new[~mask]

df_cleaned = remove_outliers_robust_zscore(df_new, columns_to_clean)
```

Применительно к нашему датасету данный метод отнес все значения угла нашивки 90^0 к выбросам, поэтому не использовался.

3. Метод межквартильного размаха (IQR) с коэффициентом multiplier с 1,5 по алгоритму:

```
def remove_outliers_iqr(df_new, columns, multiplier=1.5):
    q1 = df_new[columns].quantile(0.25)
    q3 = df_new[columns].quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - (multiplier * iqr)
    upper_bound = q3 + (multiplier * iqr)
    mask = (df_new[columns] >= lower_bound) & (df_new[columns] <= upper_bound)
    return df_new[mask]
df_cleaned = remove_outliers_iqr(df_new, columns_to_clean)
df_cleaned
```

В результате количество обнаруженных выбросов по столбцам датасета составило:

Соотношение матрица-наполнитель	6
Плотность, кг/м3	9
модуль упругости, ГПа	2
Количество отвердителя, м.%	14
Содержание эпоксидных групп,%_2	2
Температура вспышки, C_2	8
Поверхностная плотность, г/м2	2
Модуль упругости при растяжении, ГПа	6
Прочность при растяжении, МПа	11
Потребление смолы, г/м2	8
Угол нашивки, град	0
Шаг нашивки	4
Плотность нашивки	20

Обнаруженные выбросы были удалены с сокращением количества строк в датасете до 936 строк. Тем не менее, сохранился ряд крайне низких значений поверхностной плотности за пределами реальных физических характеристик материалов. В связи с этим было принято решение удалить строки со значением показателя поверхностной плотности менее 12 г/м^2 , сократив таким образом выборку до 926 строк.

Основные статистические характеристики переменных полученного после очистки датасета представлены на рисунке 8 и характеризуются меньшей зашумленностью.


```
df_cleaned = df_cleaned.drop(df_cleaned[df_cleaned['Поверхностная плотность, г/м2'] < 12].index)
df_cleaned.describe()
```

	Соотношение матрица-наполнитель	Плотность, кг/м3	модуль упругости, ГПа	Количество отвердителя, м.%	Содержание эпоксидных групп, %_2	Температура вспышки, С_2	Поверхностная плотность, г/м2	Модуль упругости при растяжении, ГПа	Прочность при растяжении, МПа	Потребление смолы, г/м2	Угол нашивки, град	n
count	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	92
mean	2.926307	1974.029569	739.551180	111.044425	22.199022	286.048775	488.139080	73.314194	2470.105431	217.660709	46.166307	
std	0.894062	70.498093	327.567455	26.971768	2.398415	39.479659	277.262832	3.048756	461.760034	57.818439	45.009193	
min	0.547391	1784.482245	2.436909	38.668500	15.695894	179.374391	12.040487	65.553336	1250.392802	63.685698	0.000000	
25%	2.320191	1923.521521	501.560124	92.619058	20.554570	259.205321	271.342426	71.225212	2150.952583	179.719792	0.000000	
50%	2.903269	1977.258043	738.847005	111.162090	22.180360	286.024118	462.275850	73.269373	2457.959767	218.697660	90.000000	
75%	3.545717	2019.842447	957.391294	130.109815	23.961585	313.012786	696.513284	75.322715	2754.226864	256.268018	90.000000	
max	5.314144	2161.565216	1649.415706	181.828448	28.955094	386.067992	1291.340115	81.417126	3705.672523	359.052220	90.000000	1

Рисунок 8 - Описательная статистика датасета после очистки данных

2.1.2 Нормализация значений переменных

Чтобы понять, насколько полученные после очистки данные близки по распределению значений к нормальному перед последующей предобработкой, целесообразно вновь обратиться к диаграммам распределения (рис.9). Визуально часть переменных подчиняется нормальному закону распределения.

```
df_cleaned.columns
column_names = df_cleaned.columns
a = 5 # количество строк
b = 5 # количество столбцов
c = 1 # запуск plot counter
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize = (35,35))
plt.suptitle('Гистограммы распределения изучаемых переменных-характеристик после очистки', fontsize = 25)
for col in df_cleaned.columns:
    plt.subplot(a, b, c)
    sns.histplot(data = df_cleaned[col], kde=True, color = "darkgreen")
    plt.ylabel(None)
    plt.title(col, size = 20)
    c += 1
```

Гистограммы распределения переменных после очистки

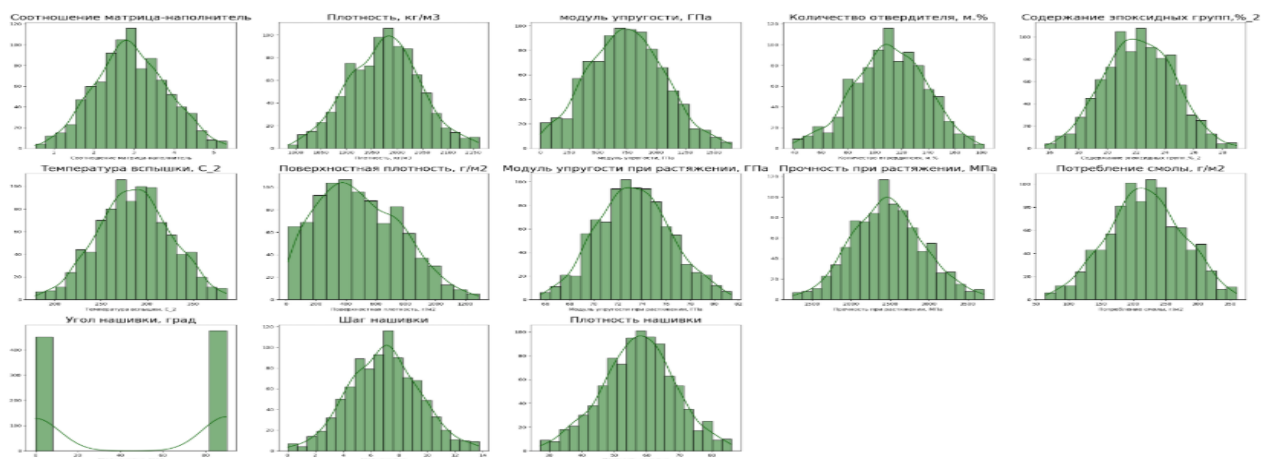


Рисунок 9 - Распределение значений переменных после очистки данных

Однако, в отношении таких переменных, как модуль упругости, поверхностная плотность, количество отвердителя, плотность нашивки, угол нашивки имеет место иной тип распределения (например, логнормальный).

Для проверки нормальности распределения очищенных данных по указанным переменным в работе использован тест Шапиро-Уилка.

```
from scipy.stats import shapiro
data = df_cleaned["модуль упругости, ГПа"]
def shapiro_test(data, alpha=0.05):
    stat, p = shapiro(data)
    if p > alpha:
        print('Данные выглядят как нормальные (гауссовские)')
    else:
        print('Данные не выглядят как нормальные (гауссовские)')
shapiro_test(df_cleaned["модуль упругости, ГПа"])
Данные не выглядят как нормальные (гауссовские)
```

Распределение данных по показателям «модуль упругости», «плотность нашивки», «поверхностная плотность», «количество отвердителя» не соответствует функции Гаусса.

Учитывая результаты проверки на нормальность распределения и разность масштабов значений переменных, необходимо прибегнуть к нормализации данных для последующего анализа, т.е. привести данные к единой шкале без искажения различий в диапазонах значений.

Для целей нормализации данных в работе использована функция MinMaxScaler, которая позволяет преобразовать признаки так, чтобы они имели одинаковый масштаб, предотвращая доминирование одних признаков над другими, с помощью алгоритма:

```
from sklearn import preprocessing
import numpy as np
scaler = MinMaxScaler()
min_max_scaler = preprocessing.MinMaxScaler()
col = df_norm.columns
result = scaler.fit_transform(df_norm)
df_norm = pd.DataFrame(min_max_scaler.fit_transform(df_norm), columns=df_norm.columns)
print(df_norm.head())
df_norm.describe()
```

Описательная статистика датасета после нормализации данных приведена на рисунке 10.

	Соотношение матрица-наполнитель	Плотность, кг/м3	модуль упругости, ГПа	Количество отвердителя, м.%	Содержание эпоксидных групп, %_2	Температура вспышки, С_2	Поверхностная плотность, г/м2	Модуль упругости при растяжении, ГПа	Прочность при растяжении, МПа	Потребление смолы, г/м2	Угол нашивки, град	на
count	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926.000000	926
mean	0.499064	0.502667	0.447555	0.505560	0.490462	0.516099	0.372156	0.489218	0.496771	0.521302	0.512959	0
std	0.187562	0.186956	0.198890	0.188403	0.180887	0.191006	0.216730	0.192183	0.188068	0.195751	0.500102	0
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0
25%	0.371909	0.368723	0.303054	0.376855	0.366438	0.386228	0.202691	0.357536	0.366785	0.392848	0.000000	0
50%	0.494231	0.511229	0.447128	0.506382	0.489054	0.515980	0.351939	0.486393	0.491825	0.524812	1.000000	0
75%	0.629008	0.624160	0.579822	0.638735	0.623393	0.646553	0.535037	0.615829	0.612490	0.652011	1.000000	0
max	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1

Рисунок 10 - Основные статистические характеристики датасета после нормализации

Для графического отображения полученных распределений и взаимосвязей целесообразно использовать инструментальный Matplotlib для построения серии 3D визуализаций, линейных графиков полученных нормализованных переменных. Например, для анализа влияния независимых переменных соотношения матрица-наполнитель и угла нашивки или содержания эпоксидных групп и поверхностной плотности на целевую переменную - модуль упругости при растяжении. Для разработки моделей прогнозов во второй части работы на вход будут подаваться нормализованные значения независимых переменных, целевой показатель тренировочной части выборки будет взят в фактических значениях для получения сразу интерпретируемых результатов прогнозирования.

2.2 Разработка, обучение и тестирование моделей прогнозирования модуля упругости при растяжении

В соответствии с постановкой задачи разработка моделей прогнозирования осуществляется в отношении двух целевых переменных: модуль упругости при растяжении и прочность при растяжении. Для этого используются алгоритмы машинного обучения, описанные нами выше. Определяем зависимые и

независимую целевую переменную для получения прогноза на основе того или иного уравнения регрессии.

Ранее было определено, что значимые линейные связи между изучаемыми переменными датасета отсутствуют. Однако, это не исключает того, что может иметь место тесная нелинейная взаимосвязь. Это может привести к переобучению или плохому обобщению данных, что повлияет на прогностическую ценность будущей модели. При этом выбор оптимального набора предикторов может быть реализован путем создания нескольких вариантов набора независимых переменных в моделях.

На первом шаге в перечень независимых переменных можно включить все переменные, кроме целевых. При разделении общего объема данных на тренировочную и тестовую выборки используется объем тестовой выборки -30%:

```
x = df_norm [['Соотношение матрица-наполнитель', 'Плотность, кг/м3',  
             'модуль упругости, ГПа', 'Количество отвердителя, м.%',  
             'Содержание эпоксидных групп,%_2', 'Температура вспышки, С_2',  
             'Поверхностная плотность, г/м2', 'Потребление смолы, г/м2',  
             'Угол нашивки, град', 'Шаг нашивки', 'Плотность нашивки']]  
  
y = df_cleaned ['Модуль упругости при растяжении, ГПа']  
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.30, random_state=42)  
df_norm.shape[0] - x_train.shape[0] - x_test.shape[0]
```

Результаты моделирования с помощью «дерева решений», линейной регрессии, «случайного леса», полиномиальной регрессии, алгоритма К-ближайших соседей приведены в таблице 2.

Таким образом, полученные модели обладают низкой точностью и не могут использоваться для прогнозирования изучаемой целевой переменной. Для получения лучших результатов целесообразно попытаться сократить количество независимых переменных с учетом их взаимозависимости, чтобы не перегружать модель многомерностью пространства и поддерживать производительность, получать надёжные результаты. Выделение наиболее значимых признаков поможет повысить точность прогнозирования. Для этих целей можно

использовать библиотеку Eli5 для описания моделей машинного обучения, которая помогает объяснять вклад каждой переменной в модель.

Таблица 2- Запись и полученные метрики моделей прогнозирования модуля упругости при растяжении в первой постановке

Наименование алгоритма	Запись	Метрики
Дерево решений (Decision Tree)	<code>tree_model = DecisionTreeRegressor(random_state=23)</code> <code>tree_model.fit(x_train, y_train)</code>	MSE = 19,641 MAE = 3,433 $R^2 = -0,979$
Линейная регрессия (Linear Regression)	<code>lr_model = LinearRegression()</code> <code>lr_model.fit(x_train, y_train)</code>	MSE = 9,986 MAE = 2,601 $R^2 = -0,006$
Случайный лес (Random Forest)	<code>forest_model = RandomForestRegressor(n_estimators=100, random_state=42)</code> <code>forest_model.fit(x_train, y_train)</code>	MSE = 10,105 MAE = 2,582 $R^2 = -0,0179$
К – ближайших соседей (K Nearest Neighbors)	<code>KN_regressor = KNeighborsRegressor(n_neighbors=10)</code> <code>KN_regressor.fit(x_train, y_train)</code> <code>y_KN_pred = KN_regressor.predict(x_test)</code>	MSE = 10,312 MAE = 2,667 $R^2 = -0,0388$
Полиномиальная регрессия (Polynomial Regression)	<code>from sklearn.preprocessing import PolynomialFeatures</code> <code># Создание полиномиальных признаков</code> <code>poly = PolynomialFeatures(degree=2)</code> <code>x_poly_train = poly.fit_transform(x_train)</code> <code>x_poly_test = poly.transform(x_test)</code> <code>from sklearn.linear_model import LinearRegression</code> <code># Обучение модели</code> <code>model_L = LinearRegression()</code> <code>model_L.fit(x_poly_train, y_train)</code> <code>y_poly_pred = model_L.predict(x_poly_test)</code>	MSE = 9,880 MAE = 2,603 $R^2 = 0,0047$

Для определения ряда переменных по наибольшему влиянию на результат в линейной регрессии можно использовать алгоритм:

```
!pip install eli5
import eli5
from eli5.sklearn import PermutationImportance
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.30, random_state=42)
lr_model = LinearRegression()
lr_model.fit(x_train, y_train)
perm = PermutationImportance(lr_model, random_state=42).fit(x_test, y_test)
eli5.show_weights(perm, feature_names = x_test.columns.tolist())
```

Weight	Feature
0.0066 ± 0.0110	Содержание эпоксидных групп, % ₂
0.0026 ± 0.0068	Соотношение матрица-наполнитель
0.0014 ± 0.0005	Количество отвердителя, м. %
0.0014 ± 0.0024	Плотность нашивки
0.0007 ± 0.0006	Угол нашивки, град
0.0006 ± 0.0056	Поверхностная плотность, г/м ²
0.0002 ± 0.0009	Плотность, кг/м ³
0.0000 ± 0.0076	Потребление смолы, г/м ²
-0.0009 ± 0.0061	Температура вспышки, C ₂
-0.0013 ± 0.0014	Шаг нашивки
-0.0156 ± 0.0048	модуль упругости, ГПа

Наиболее важные независимые переменные имеют наибольшие значения показателя Weight. Такой подход дал видимое улучшение значения коэффициента детерминации в линейной регрессии, однако он остался низким.

В последующих построениях имеет смысл ограничить перечень независимых переменных 6 первыми выделенными переменными:

- содержание эпоксидных групп;
- соотношение матрица-наполнитель;
- количество отвердителя;
- плотность нашивки;
- угол нашивки;
- поверхностная плотность.

Результаты моделирования зависимости модуля упругости при растяжении от перечисленных выше переменных на примере нескольких алгоритмов представлены в таблице 3.

Наилучшие результаты показал градиентный бустинг, но, несмотря на улучшение значений метрик, в целом полученные модели регрессии не обладают требуемой точностью и не могут применяться в целях прогнозирования целевого показателя.

Создание Pipeline машинного обучения с объединением всех инструменты в один конвейер и использованием регрессии Ridge без повторяющегося кода также не дало значимый результат.

В завершение разработки прогнозной модели показателя модуль упругости при растяжении создана простая нейронная сеть, для которой круг независимых переменных ограничился выделенными ранее наиболее значимыми 6 параметрами. Нейронная сеть работает на базе последовательной модели Sequential, включает 4 полносвязных слоя Dense, встроенную функцию нормализации данных BatchNormalization. Обучение модели осуществляется на условиях: количество эпох 100, партии выборок переменных для обучения – по 15,

предусмотрена ранняя остановка при отсутствии улучшений после 7 эпох и выводом метрик после каждой эпохи.

Таблица 3- Запись и полученные метрики ряда моделей прогнозирования модуля упругости при растяжении после выделения 6 независимых переменных

Наименование алгоритма	Запись	Метрики
Линейная регрессия (Linear Regression)	<pre> x_reduced = df_norm[['Соотношение матрица-наполнитель', 'Количество отвердителя, м.%', 'Содержание эпоксидных групп, %2', 'Поверхностная плотность, г/м2', 'Угол нашивки, град', 'Плотность нашивки']] y_reduced = df_cleaned['Модуль упругости при растяжении, ГПа'] x_train_reduced, x_test_reduced, y_train_reduced, y_test_reduced = train_test_split(x_reduced, y_reduced, test_size=0.30, random_state=42) # Посмотрим, как работает модель на сокращенном наборе переменных # Модель линейной регрессии lr_model2 = LinearRegression() lr_model2.fit(x_train_reduced, y_train_reduced) y_lr_model_test_pred2 = lr_model2.predict(x_test_reduced) </pre>	MSE = 9,863 MAE = 2,582 R² = 0,0064
Градиентный бустинг (GBA)	<pre> GBR_model = GradientBoostingRegressor(loss='absolute_error', learning_rate=0.1, n_estimators=300, max_depth = 1, random_state = 23, max_features = 5) GBR_model.fit(x_train_reduced, y_train_reduced) GBR_pred = GBR_model.predict(x_test_reduced) </pre>	MSE = 9,782 MAE = 2,553 R² = 0,0146
Регрессия опорных векторов (SVR) с функциями ядра: базисной	<pre> svr_rbf = SVR(kernel="rbf", C=100, gamma=0.1, epsilon=0.1) svr_lin = SVR(kernel="linear", C=100, gamma="auto") svr_poly = SVR(kernel="poly", C=100, gamma="auto", degree=4, epsilon=0.1, coef0=1) svrs_model = [svr_rbf, svr_lin, svr_poly] </pre>	MSE = 10,084 MAE = 2,606 R² = - 0,0016
линейной	<pre> for svr in svrs_model: svr.fit(x_train_reduced, y_train_reduced) svr_rbf_pred = svr_rbf.predict(x_test_reduced) svr_lin_pred = svr_lin.predict(x_test_reduced) svr_poly_pred = svr_poly.predict(x_test_reduced) </pre>	MSE = 9,988 MAE = 2,582 R² = -0,0062
полиномиальной		MSE = 10,220 MAE = 2,616 R² = -0,0296

В нейронную сеть встроена регуляризация L₂ с коэффициентом 0,003 – техника, которая помогает контролировать сложность модели и защищать её от переобучения, уменьшая на введенный коэффициент удельные веса признаков в модели. Доля данных, которые должны быть зарезервированы для валидации (validation_split)-20%:


```

x_N = df_cleaned [['Соотношение матрица-наполнитель', 'Количество отвердителя, м.%',
                  'Содержание эпоксидных групп,%_2',
                  'Поверхностная плотность, г/м2',
                  'Угол нашивки, град', 'Плотность нашивки']]

y_N = df_cleaned ['Модуль упругости при растяжении, ГПа']
x_trainN, x_testN, y_trainN, y_testN = train_test_split(x_N, y_N, test_size=0.30, random_state=42)

from keras.models import Sequential
from keras.layers import Dense
from tensorflow.keras import layers
from keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping

model= Sequential()
model.add(Dense(128, activation='relu', input_shape=(x_trainN.shape[1],), kernel_regularizer= l2(0.003)))
model.add(BatchNormalization())
model.add(Dense(256, activation='relu', kernel_regularizer= l2(0.003)))
model.add(BatchNormalization())
model.add(Dense(64, activation='relu', kernel_regularizer= l2(0.003)))
model.add(BatchNormalization())
model.add(Dense(1, activation='linear'))

model.compile(optimizer='adam', loss='mse', metrics=[tf.keras.metrics.R2Score()])
early_stopping = EarlyStopping(monitor='val_loss', patience=7)

# Обучение модели на условиях : количество эпох 100, партии выборки переменных для обучения - по 15,
# ранняя остановка при отсутствии улучшений после 7 эпох и выводом метрик после каждой эпохи.
model.fit(x_trainN, y_trainN, epochs=int(100), batch_size=15, validation_split=0.2, verbose=1, callbacks=[early_stopping])
    
```

Нейронная сеть с такой архитектурой обладает в нашем случае низкой предсказательной способностью. Наглядно соотношение фактических и полученных посредством нейронной сети прогнозных значений целевой переменной представлены на рисунке 11.

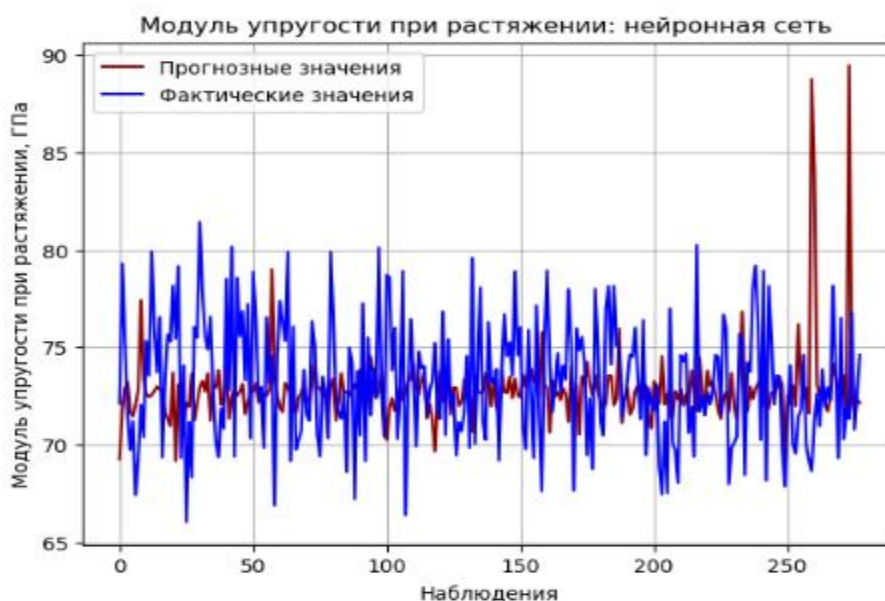


Рисунок 11- Фактические и спрогнозированные нейронной сетью показатели модуля упругости при растяжении

2.3 Разработка, обучение и тестирование моделей прогнозирования прочности при растяжении

Прогнозирование прочности при растяжении осуществляется в той же последовательности. На первом этапе обработки данных с использованием функций моделирования с помощью линейной регрессии, «случайного леса», «дерева решений», полинома 2 степени, К-ближайших соседей получены низкие значения метрик точности моделей, не позволяющие использовать их для получения прогнозов прочности при растяжении.

Путем обращения к библиотеке Eli5 выделены 6 наиболее значимых независимых переменных, обуславливающих большую часть изменений прочности при растяжении, в том числе:

- плотность;
- количество отвердителя;
- шаг нашивки;
- поверхностная плотность;
- модуль упругости;
- потребление смолы.

```
import eli5
from eli5.sklearn import PermutationImportance
x1_train, x1_test, y1_train, y1_test = train_test_split(x1, y1, test_size=0.30, random_state=42)
lr_model = LinearRegression()
lr_model.fit(x1_train, y1_train)
perm = PermutationImportance(lr_model, random_state=42).fit(x1_test, y1_test)
eli5.show_weights(perm, feature_names = x1_test.columns.tolist())
```

Weight	Feature
0.0139 ± 0.0099	Плотность, кг/м3
0.0081 ± 0.0132	Количество отвердителя, м.%
0.0050 ± 0.0100	Шаг нашивки
0.0017 ± 0.0073	Поверхностная плотность, г/м2
0.0013 ± 0.0018	модуль упругости, ГПа
0.0005 ± 0.0004	Потребление смолы, г/м2
0.0003 ± 0.0009	Температура вспышки, C_2
-0.0002 ± 0.0055	Угол нашивки, град
-0.0006 ± 0.0027	Содержание эпоксидных групп, %_2
-0.0020 ± 0.0025	Соотношение матрица-наполнитель
-0.0057 ± 0.0127	Плотность нашивки

Таким образом, в процессе дальнейшей работы моделирование осуществляется исходя из данного перечня независимых переменных с результатами, отраженными в таблице 4.

Таблица 4 - Запись и полученные метрики ряда моделей прогнозирования прочности при растяжении после выделения 6 независимых переменных

Наименование алгоритма	Запись	Метрики
Линейная регрессия (Linear Regression)	<pre>x1_reduced = df_norm [['плотность, кг/м3', 'модуль упругости, ГПа', 'Количество отвердителя, м.%', 'Потребление смолы, г/м2', 'Поверхностная плотность, г/м2', 'Шаг нашивки']] y1_reduced = df_cleaned ['прочность при растяжении, МПа'] x1_train_reduced, x1_test_reduced, y1_train_reduced, y1_test_reduced = train_test_split(x1_reduced, y1_reduced, test_size=0.30, random_state=42) # Посмотрим, как работает модель на сокращенном наборе переменных # Модель линейной регрессии lr1_model = LinearRegression() lr1_model.fit(x1_train_reduced, y1_train_reduced) y1_lr_model_pred = lr1_model.predict(x1_test_reduced)</pre>	MSE = 202612,3 MAE = 359,5 $R^2 = 0,022$
Полиномиальная регрессия (Polynomial Regression)	<pre>poly = PolynomialFeatures(degree=2) x1_poly_train = poly.fit_transform(x1_train_reduced) x1_poly_test = poly.transform(x1_test_reduced) from sklearn.linear_model import LinearRegression # Обучение модели model = LinearRegression() model.fit(x1_poly_train, y1_train_reduced) y1_poly_pred = model.predict(x1_poly_test)</pre>	MSE = 203322,8 MAE = 366,61 $R^2 = 0,0186$
Градиентный бустинг (GBA)	<pre>GBR1_model = GradientBoostingRegressor(loss='absolute_error', learning_rate=0.1, n_estimators=300, max_depth = 1, random_state = 42, max_features = 5) GBR1_model.fit(x1_train_reduced, y1_train_reduced) GBR1_pred = GBR1_model.predict(x1_test_reduced)</pre>	MSE = 208489,6 MAE = 367,5 $R^2 = - 0,0063$
Регрессия опорных векторов (SVR) с функциями ядра: базисной	<pre>svr1_rbf = SVR(kernel="rbf", C=100, gamma=0.1, epsilon=0.1) svr1_lin = SVR(kernel="linear", C=100, gamma="auto") svr1_poly = SVR(kernel="poly", C=100, gamma="auto", degree=4, epsilon=0.1, coef0=1) svrs1_model = [svr1_rbf, svr1_lin, svr1_poly]</pre>	MSE = 203693,0 MAE = 358,8 $R^2 = 0,0168$
линейной	<pre>for svr1 in svrs1_model: svr1.fit(x1_train_reduced, y1_train_reduced) svr1_rbf_pred = svr1_rbf.predict(x1_test_reduced) svr1_lin_pred = svr1_lin.predict(x1_test_reduced) svr1_poly_pred = svr1_poly.predict(x1_test_reduced)</pre>	MSE = 202164,7 MAE = 260,5 $R^2 = 0,0242$
полиномиальной		MSE = 201520,0 MAE = 361,0 $R^2 = 0,0273$

Наилучшее значение метрик получено для SVR-модели, линейной регрессии, схожие результаты показало использование полинома 2 степени. Однако, как и в случае с прогнозированием модуля упругости при растяжении, ни одна из моделей не имеет достаточной точности для разработки прогнозов.

Аналогичная по архитектуре нейронная сеть на базе последовательной модели Sequential с 5 полносвязными слоями Dense и встроенной нормализацией за 200 эпох обучения также показала низкую точность прогноза (рис.12).

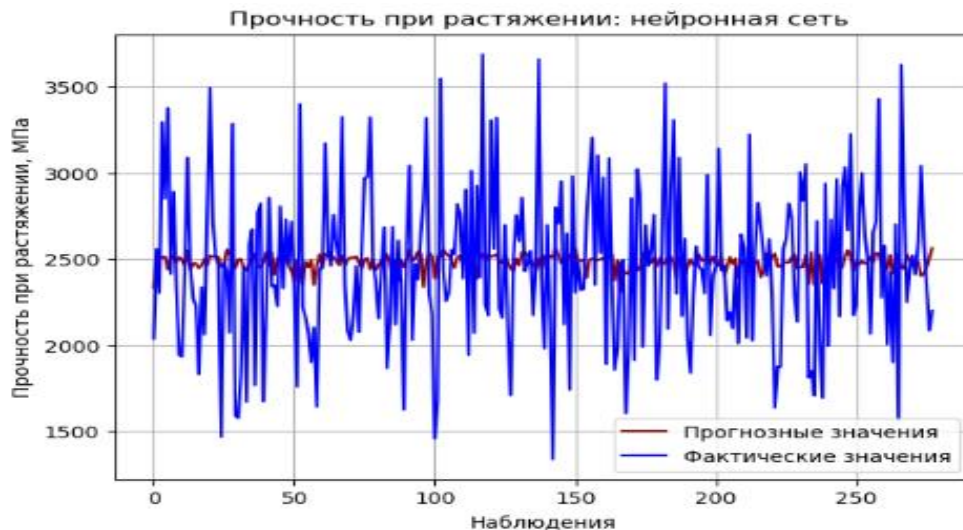


Рисунок 12 - Фактические и спрогнозированные нейронной сетью показатели прочности при растяжении

2.4 Создание нейронной сети для рекомендации соотношения

«матрица – наполнитель»

В качестве зависимой целевой переменной выступает соотношение «матрица-наполнитель». Тестовая выборка в объеме 30% общего массива данных. Для нормализации данных используется пакетный нормализатор, активация осуществляется, как и в нейронных сетях, построенных ранее, с помощью функции Relu. Эта кусочно-линейная функция, которая выводит непосредственно входные данные, если они положительны; в противном случае она выводит ноль. Relu позволяет положительным значениям проходить через функцию без изменений, а все отрицательные значения обнуляет. Это помогает нейронной сети сохранять необходимую сложность для изучения закономерностей и при этом избегать некоторых проблем, связанных с другими функциями активации, таких как проблема исчезающего градиента.

Для получения рекомендации вводятся значения следующих показателей-переменных модели:

```

xs = df_cleaned [['Прочность при растяжении, МПа', 'Модуль упругости при растяжении, ГПа', 'Количество отвердителя, м.%',
                  'Плотность, кг/м3', 'Поверхностная плотность, г/м2', 'Содержание эпоксидных групп, %2',
                  'Угол нашивки, град', 'Плотность нашивки']]

ys = df_cleaned ['Соотношение матрица-наполнитель']

x_trainS, x_testS, y_trainS, y_testS = train_test_split(xs, ys, test_size=0.30, random_state=42)

from keras.models import Sequential
from keras.layers import Dense
from tensorflow.keras import layers
from keras.regularizers import l2
from tensorflow.keras.callbacks import EarlyStopping
# В архитектуре нейронной сети используются простые слои Dense и встроенный пакетный нормализатор BatchNormalization:
models = Sequential()
models.add(Dense(128, activation='relu', input_shape=(x_trainS.shape[1],), kernel_regularizer=l2(0.003)))
models.add(BatchNormalization())
models.add(Dense(256, activation='relu', kernel_regularizer=l2(0.003)))
models.add(BatchNormalization())
models.add(Dense(64, activation='relu', kernel_regularizer=l2(0.003)))
models.add(BatchNormalization())
models.add(Dense(1, activation='relu'))

models.compile(optimizer='adam', loss='mse', metrics=[tf.keras.metrics.R2Score()])
early_stopping = EarlyStopping(monitor='val_loss', patience=7)
# Вывод параметров модели
models.summary()
# Обучение модели на условиях: количество эпох 200, партии выборки переменных для обучения - по 12, ранняя остановка при отсутствии улучшений
# после 7 эпох и выводом метрик после каждой эпохи.
models.fit(x_trainS, y_trainS, epochs=int(200), batch_size=12, validation_split=0.2, verbose=1, callbacks=[early_stopping])
    
```

Обучение модели идет на условиях: 200 эпох, партии выборки переменных для обучения - по 12, ранняя остановка при отсутствии улучшений после 7 эпох и выводом метрик после каждой эпохи:

Визуализация фактических (тестовый набор) и прогнозных значений целевой переменной отражена на рисунке 13.

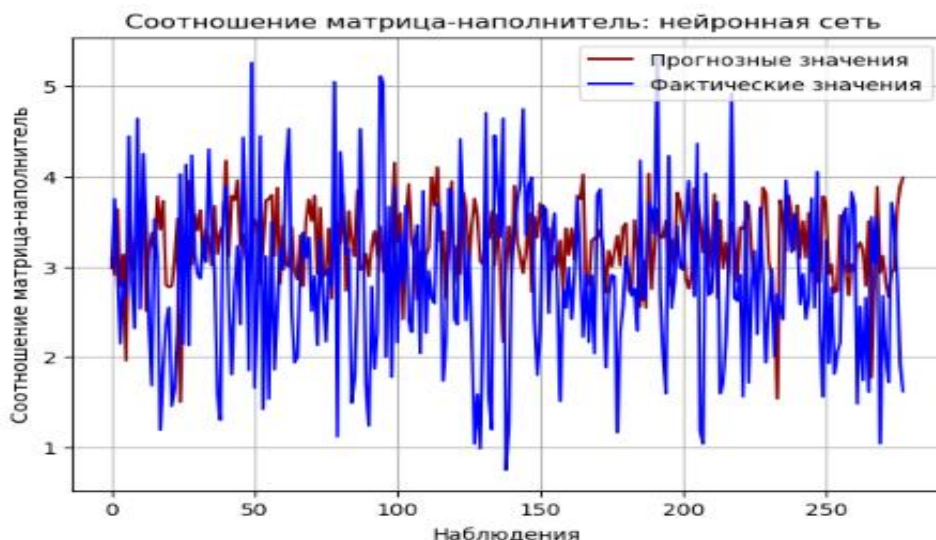


Рисунок 13- Фактические и предсказанные нейронной сетью значения соотношения «матрица-наполнитель»

2.5 Разработка веб-приложения

На основе созданной нейронной сети с целью получения рекомендаций по соотношению «матрица-наполнитель» при разработке композиционных материалов с требуемыми прочностными характеристиками, а также заданными параметрами их компонентов разработано веб-приложение, имеющие понятный простой интерфейс калькулятора. Для расчета прогнозного значения необходимо ввести значения 8 переменных:

- прочность при растяжении, МПа,
- модуль упругости при растяжении, ГПа,
- количество отвердителя, м.%,
- плотность, кг/м³,
- поверхностная плотность, г/м²,
- содержание эпоксидных групп, %2,
- угол нашивки, град,
- плотность нашивки.

Приложение разработано в Visual Studio, интегрированной среде разработки (IDE), с использованием шаблона веб-проекта Flask. Основу файловой организации приложения составляет файл «app.py» с кодами, запускающими сервер Flask, осуществляющими загрузку модели из ранее сохраненного файла «model.keras» и получение 8 переменных для обработки. Интерфейс приложения поддерживается за счет обращения к файлу «index.html» (рис.14).

Функционал Visual Studio позволяет сделать при необходимости корректировки кодов в данных файлах, собрать и пересобрать проект заново и запустить его с адресов:

Serving Flask app 'app'

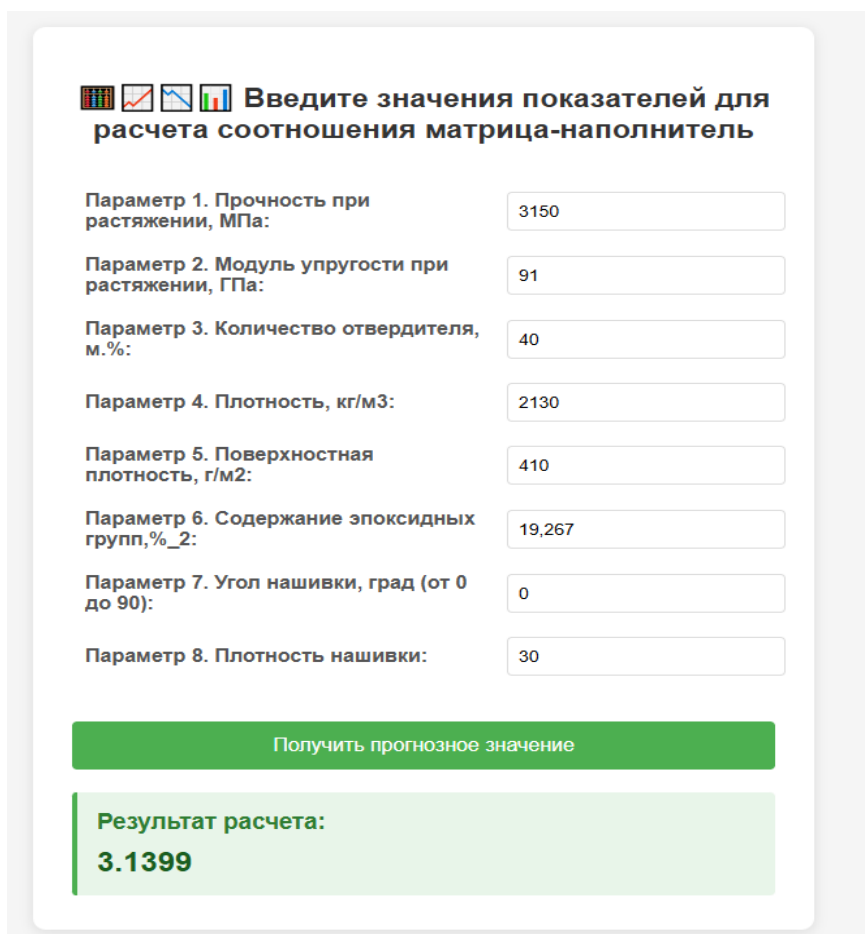
* Debug mode: on

WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

* Running on <http://192.168.1.48:5000>



Введите значения показателей для расчета соотношения матрица-наполнитель

Параметр 1. Прочность при растяжении, МПа:	3150
Параметр 2. Модуль упругости при растяжении, ГПа:	91
Параметр 3. Количество отвердителя, м. %:	40
Параметр 4. Плотность, кг/м3:	2130
Параметр 5. Поверхностная плотность, г/м2:	410
Параметр 6. Содержание эпоксидных групп, %_2:	19,267
Параметр 7. Угол нашивки, град (от 0 до 90):	0
Параметр 8. Плотность нашивки:	30

Получить прогнозное значение

Результат расчета:
3.1399

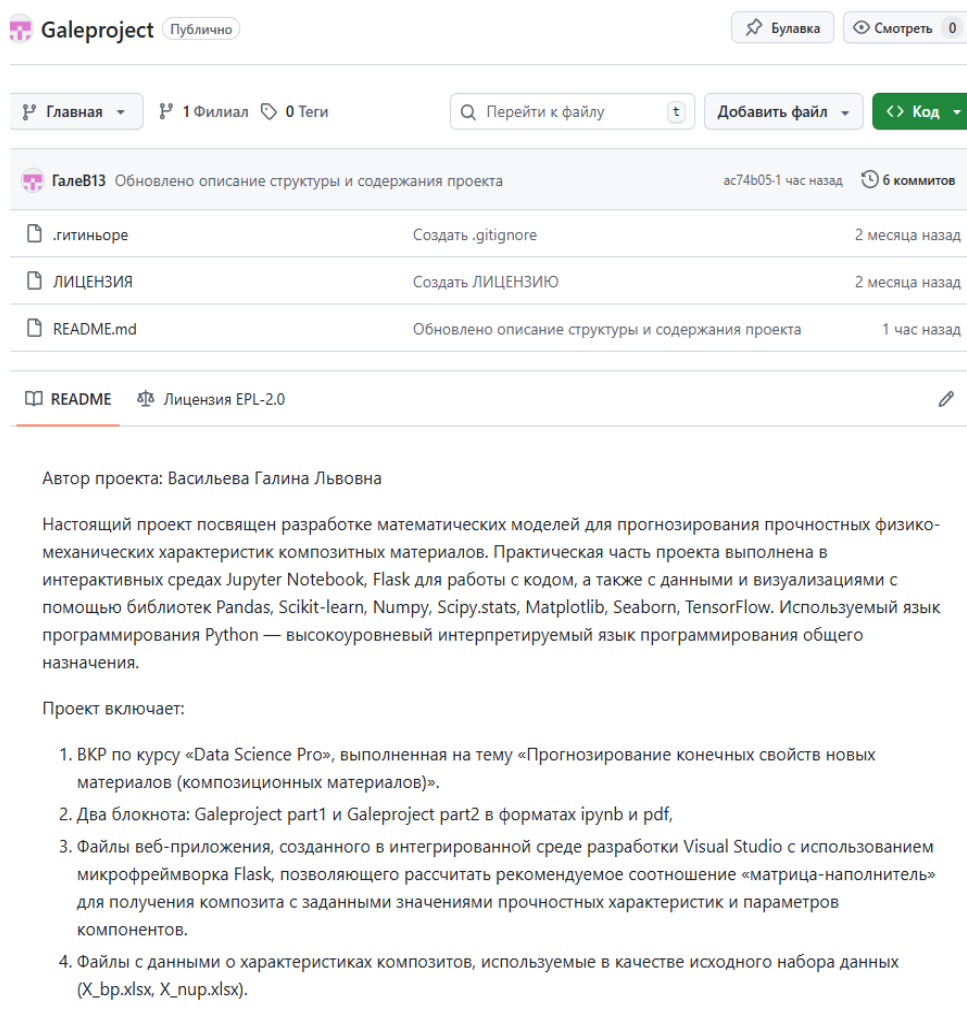
Рисунок 14- Интерфейс веб-приложения для расчета рекомендаций соотношения матрица-наполнитель в композиционном материале

2.6 Создание удалённого репозитория и загрузка результатов

Хранения и управления файлами проекта создается удаленный репозиторий. Для работы с репозиторием используются специальные инструменты, такие как Git. Git – это система контроля версий, которая позволяет командам разработчиков совместно работать над проектом, создавать и управлять репозиториями. С помощью Git можно создавать новые ветки разработки, объединять изменения из разных веток, восстанавливать предыдущие версии файлов и многое другое.

Репозиторий был создан на github.com по адресу:

<https://github.com/GaleV13/Galeproject> (рис.15)



The screenshot shows a GitHub repository named 'Galeproject' which is public. The repository has 1 branch and 0 tags. The commit history shows three recent commits: creating a .gitignore file, creating a LICENSE file, and updating the README and project structure. The README file is selected, showing the project description in Russian. The author is Galina Lyubovna Vasileva. The project is dedicated to developing mathematical models for predicting the mechanical characteristics of composite materials. It includes a Jupyter Notebook, a Flask web application, and data files. The project is part of a course 'Data Science Pro'.

ГалеВ13 Обновлено описание структуры и содержания проекта ас74b05-1 час назад 6 коммитов

Файл	Описание	Время
.gitignore	Создать .gitignore	2 месяца назад
ЛИЦЕНЗИЯ	Создать ЛИЦЕНЗИЮ	2 месяца назад
README.md	Обновлено описание структуры и содержания проекта	1 час назад

README Лицензия EPL-2.0

Автор проекта: Васильева Галина Львовна

Настоящий проект посвящен разработке математических моделей для прогнозирования прочностных физико-механических характеристик композитных материалов. Практическая часть проекта выполнена в интерактивных средах Jupyter Notebook, Flask для работы с кодом, а также с данными и визуализациями с помощью библиотек Pandas, Scikit-learn, Numpy, Scipy.stats, Matplotlib, Seaborn, TensorFlow. Используемый язык программирования Python — высокоуровневый интерпретируемый язык программирования общего назначения.

Проект включает:

1. ВКР по курсу «Data Science Pro», выполненная на тему «Прогнозирование конечных свойств новых материалов (композиционных материалов)».
2. Два блокнота: Galeproject part1 и Galeproject part2 в форматах ipynb и pdf,
3. Файлы веб-приложения, созданного в интегрированной среде разработки Visual Studio с использованием микрофреймворка Flask, позволяющего рассчитать рекомендуемое соотношение «матрица-наполнитель» для получения композита с заданными значениями прочностных характеристик и параметров компонентов.
4. Файлы с данными о характеристиках композитов, используемые в качестве исходного набора данных (X_bp.xlsx, X_nup.xlsx).

Рисунок 15- Фрагмент страницы репозитория

Наполнением репозитория стали созданные файлы практической части работы, файлы веб-приложения, пояснительная записка (ВКР, включающая теоретическую, аналитическую и практическую части).

Заключение

В рамках настоящей исследовательской работы изучены теоретические основы и предложены практические подходы к разработке моделей прогнозирования целевых показателей объекта на примере моделирования прочностных характеристик композиционных материалов.

С помощью современных инструментов программной обработки больших массивов данных на языке Python с использованием библиотек Pandas, Scikit-learn, Numpy, Scipy.stats, Matplotlib, Seaborn, TensorFlow проведен разведочный анализ данных о физико-механических свойствах композитных материалов и их компонентов (массив 13 столбцов, 1023 строки), их предобработка и построение моделей машинного обучения, включая нейронные сети.

По итогам моделирования наилучшие результаты по метрикам полученных моделей прогнозирования модуля упругости при растяжении композитного материала имеет градиентный бустинг, показателя прочности при растяжении - SVR -регрессия, полином второй степени. Созданы алгоритмы простых 4-5-тислойных нейронных сетей для получения прогнозов, один из которых для рекомендаций соотношения «матрица-наполнитель» в зависимости от значений 8 параметров композита и его компонентов положен в основу разработки веб-приложения -калькулятора прогнозов.

Модели в целом характеризуются низкой точностью и на данном этапе не представляют, таким образом, прогностической ценности в связи с возможным наличием синтетических данных в первоначальном массиве, их недостаточностью, отсутствием части информации у исследователя об опытных образцах.

Однако, сами методические подходы и алгоритмы поиска решений с учетом более глубокого знания физики вопроса создания новых композиционных материалов могут быть дополнены и использоваться на практике.

Результаты исследовательской работы представлены в созданном репозитории на GitHub.

Библиографический список

1. Браунли, Дж. Подготовка данных для машинного обучения [Текст]: учебное пособие / Дж. Браунли; перевод с англ. Е. С. Капацца ; под ред. А. В. Лебедева . – М.: Издательство МГУ, 2021 . – 250 с.
2. Бурков Андрей. Машинное обучение без лишних слов [Текст]: - СПб.: Питер, 2020. -192 с.
3. Васильев, А. Н. Программирование на Python в примерах и задачах. [Текст]: /Алексей Васильев. -М.: Эксмо, 2024. -616с.
4. Ватьян А.С., Гусарова Н.Ф., Добренко Н.В. DATA SCIENCE: проблемы и решения [Текст]: - СПб: Университет ИТМО, 2025. – 219 с.
5. Вьюгин, В.В. Математические основы машинного обучения и прогнозирования [Текст]: - М.: 2018. - 484 с.
6. Григорьев Алексей. Машинное обучение. Портфолио реальных проектов [Текст]: - СПб.: Питер, 2023. - 496 с.
7. Гусев Е.Л., Черных В.Д. Перспективные подходы и методы к решению проблемы прогнозирования определяющих характеристик композиционных материалов. [Электронный ресурс]: – Режим доступа: <https://cyberleninka.ru/article/n/perspektivnye-podhody-i-metody-k-resheniyu-problemy-prognozirovaniya-opredelyayuschih-harakteristik-kompozitsionnyh-materialov/viewer> (дата обращения: 20.09.2025)
8. Двучичанская Н.Н., Слынько Л.Е., Пясецкий В.Б. Композиционные материалы. Физико-химические свойства [Текст]: Учеб, пособие. — М.: Изд-во МГТУ им. Н.Э. Баумана, 2008. - 48 с.
9. Жерон Орельен – Прикладное машинное обучение с помощью Scikit-Learn и TensorFlow [Текст] : учебник / Орельен Жерон ; пер. с англ. А. Кузнецова и др. – М.: ДМК Пресс, 2018. – 576 с.
10. Китов, В.В. Машинное и глубокое обучение. Онлайн-учебник [Электронный ресурс]: – Режим доступа: <https://deeplearning.ru/?ysclid=milr86ynyf635256749> (дата обращения: 10.09.2025)
11. Композиционные материалы [Текст]: учебник для вузов / под ред. А. А. Берлина. – М: Высшая школа, 2007. – 543 с.
12. Любимова О.Н., Любимов Е.В., Андреев В.В. Армирующие элементы на основе композита: анализ целесообразности их внедрения [Текст]:

учебное пособие для вузов / Пол1 ДВФУ. - Владивосток: Изд-во Дальневост. федерал. ун-та, 2023. –117 с.

13. Марк Лутц. Изучаем Python [Текст]: учебник для начинающих и профессионалов / Марк Лутц. – Москва: Вильямс, 2019. – 1504 с.

14. П. Брюс, Э. Брюс. Разведочный анализ данных [Текст]: учебник / П. Брюс, Э. Брюс. – СПб.: БХВ-Петербург, 2018. – 304 с.

15. Поздняков, С. Статистика, R и анализ данных [Текст]: учебное пособие / С. Поздняков; под ред. Е. С. Капацца. – Москва: Издательство МГУ, 2021. – 300 с.

16. Рындина, Светлана Валентиновна. Базовые возможности языка Python для анализа данных [Текст]: учеб.-метод. пособие / С. В. Рындина. - Пенза: Изд-во ИГУ, 2022. - 72 с.

17. Уатт, Дж. Машинное обучение: основы, алгоритмы и практика применения [Текст]: Пер. с англ. / Дж. Уатт, Р. Борхани, А. Катсаггелос. - СПб.: БХВ-Петербург, 2022. - 640 с.

18. Учебник по машинному обучению. Яндекс-образование [Электронный ресурс]: – Режим доступа: <https://education.yandex.ru/handbook/ml> (дата обращения: 15.09.2025).

19. Эрик Мэтиз. Изучаем Python. Программирование игр, визуализация данных, веб-приложения [Текст]: учебник для начинающих / Эрик Мэтиз. – Москва: Питер, 2019. – 1280 с.

20. База данных свойств материалов. MatWeb (Онлайн-справочник) [Электронный ресурс]: – Режим доступа: <https://matweb.com/search/DataSheet.aspx?MatGUID=f1436e63c0a44e80a6f116cb87cbbefb&ckck=1>

21. ГОСТР ИСО 16269-4-2017. Статистические методы. Статистическое представление данных. Часть 4. Выявление и обработка выбросов (ISO 16269-4:2010, Statistical interpretation of data - Part 4: Detection and treatment of outliers, IDT) [Текст]. – Утв. и введен в действие Приказом Федерального агентства по техническому регулированию и метрологии от 10 августа 2017 г. № 865-ст.

22. ГОСТ 32794-2014 Межгосударственный стандарт. Композиты полимерные. Термины и определения. [Электронный ресурс]: – Режим доступа: <https://internet-law.ru/gosts/gost/58766/> (дата обращения: 14.09.2025)